results

sa revision simple 30 epochs efficient net

python ./main.py \
--model efficientnet_b0 \
--mode saRevisionSimple \
--epoch 30 \
--save_path cifar10_results_30epochs/efficientnet_b0 \
--dataset cifar10 \
--batch_size 32 \
--start_revision 0 \
--task classification \
--threshold 0.3 \
--seed 123
Using device: mps
Apple Silicon GPU (MPS) enabled.
/Users/amar/Developer/energy_eff_training/venv/lib/python3.13/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
entry = pickle.load(f, encoding="latin1")

Epoch [1/30] α=2.00 T=1.0000 subset=50000
/Users/amar/Developer/energy_eff_training/saRevisionSimple.py:245: UserWarning: The epoch parameter in `scheduler.step()` was not necessary and is being deprecated where possible. Please use `scheduler.step()` to step the scheduler. During the deprecation, if epoch is different from None, the closed form is used instead of the new chainable form, where available. Please open an issue if you are unable to replicate your use case: https://github.com/pytorch/pytorch/issues/new/choose.
scheduler.step(val_loss)
TrainAcc=0.48 TrainLoss=1.4251 | TestAcc=0.6557 TestLoss=0.9581
[SA] ACCEPT δ=+0.48 α→1.00
Epoch time: 196.31s

Epoch [2/30] α=1.00 T=0.8532 subset=33333
TrainAcc=0.68 TrainLoss=0.9140 | TestAcc=0.7309 TestLoss=0.7706

[SA] ACCEPT δ=+0.20 α→2.00
Epoch time: 143.23s

Epoch [3/30] α=2.00 T=0.7279 subset=30000
TrainAcc=0.75 TrainLoss=0.7299 | TestAcc=0.7648 TestLoss=0.6777

[SA] ACCEPT δ=+0.07 α→3.00
Epoch time: 132.58s

Epoch [4/30] α=3.00 T=0.6210 subset=28571
TrainAcc=0.79 TrainLoss=0.6190 | TestAcc=0.8010 TestLoss=0.5852

[SA] ACCEPT δ=+0.04 α→2.00
Epoch time: 128.22s

Epoch [5/30] α=2.00 T=0.5298 subset=21428
TrainAcc=0.81 TrainLoss=0.5480 | TestAcc=0.7964 TestLoss=0.5887

[SA] ACCEPT δ=+0.02 α→3.00
Epoch time: 105.75s

Epoch [6/30] α=3.00 T=0.4520 subset=22222
TrainAcc=0.83 TrainLoss=0.5041 | TestAcc=0.8135 TestLoss=0.5415

[SA] ACCEPT δ=+0.02 α→2.00
Epoch time: 108.38s

Epoch [7/30] α=2.00 T=0.3857 subset=16666
TrainAcc=0.84 TrainLoss=0.4607 | TestAcc=0.8231 TestLoss=0.5178

[SA] ACCEPT δ=+0.01 α→1.00
Epoch time: 90.39s

Epoch [8/30] α=1.00 T=0.3290 subset=11111
TrainAcc=0.85 TrainLoss=0.4350 | TestAcc=0.8288 TestLoss=0.5054

[SA] ACCEPT δ=+0.01 α→0.10
Epoch time: 73.19s

Epoch [9/30] α=0.10 T=0.2807 subset=6043
TrainAcc=0.85 TrainLoss=0.4218 | TestAcc=0.8361 TestLoss=0.5022

[SA] Metropolis: δ=+0.00 r=0.81 boltz=1.00
[SA] ACCEPT δ=+0.00 α→0.10
Epoch time: 56.76s

Epoch [10/30] α=0.10 T=0.2395 subset=5445
TrainAcc=0.87 TrainLoss=0.4075 | TestAcc=0.8366 TestLoss=0.5043

[SA] ACCEPT δ=+0.02 α→0.10
Epoch time: 55.35s

Epoch [11/30] α=0.10 T=0.2043 subset=4954
TrainAcc=0.86 TrainLoss=0.4122 | TestAcc=0.8380 TestLoss=0.5007

[SA] Metropolis: δ=-0.01 r=0.28 boltz=0.95
[SA] ACCEPT δ=-0.01 α→0.10
Epoch time: 53.20s

Epoch [12/30] α=0.10 T=0.1743 subset=4545
TrainAcc=0.86 TrainLoss=0.4045 | TestAcc=0.8254 TestLoss=0.5427

[SA] Metropolis: δ=+0.00 r=0.82 boltz=1.00
[SA] ACCEPT δ=+0.00 α→1.10
Epoch time: 52.32s

Epoch [13/30] α=1.10 T=0.1487 subset=7446
TrainAcc=0.87 TrainLoss=0.3757 | TestAcc=0.8307 TestLoss=0.5057

[SA] ACCEPT δ=+0.01 α→2.10
Epoch time: 61.07s

Epoch [14/30] α=2.10 T=0.1269 subset=9627
TrainAcc=0.87 TrainLoss=0.3770 | TestAcc=0.8390 TestLoss=0.4803

[SA] Metropolis: δ=+0.00 r=0.51 boltz=1.00
[SA] ACCEPT δ=+0.00 α→3.10
Epoch time: 67.69s

Epoch [15/30] α=3.10 T=0.1083 subset=11325
TrainAcc=0.87 TrainLoss=0.3553 | TestAcc=0.8431 TestLoss=0.4721

[SA] Metropolis: δ=+0.00 r=0.82 boltz=1.00
[SA] ACCEPT δ=+0.00 α→2.10
Epoch time: 73.44s

Epoch [16/30] α=2.10 T=0.0924 subset=8563
TrainAcc=0.88 TrainLoss=0.3430 | TestAcc=0.8453 TestLoss=0.4741

[SA] ACCEPT δ=+0.01 α→1.10
Epoch time: 64.79s

Epoch [17/30] α=1.10 T=0.0788 subset=5801
TrainAcc=0.89 TrainLoss=0.3281 | TestAcc=0.8390 TestLoss=0.5032

[SA] ACCEPT δ=+0.01 α→2.10
Epoch time: 56.23s

Epoch [18/30] α=2.10 T=0.0672 subset=7711
TrainAcc=0.88 TrainLoss=0.3436 | TestAcc=0.8569 TestLoss=0.4434

[SA] Metropolis: δ=-0.01 r=0.74 boltz=0.86
[SA] ACCEPT δ=-0.01 α→1.10
Epoch time: 62.20s

Epoch [19/30] α=1.10 T=0.0574 subset=5223
TrainAcc=0.89 TrainLoss=0.3355 | TestAcc=0.8433 TestLoss=0.4812

[SA] ACCEPT δ=+0.01 α→0.10
Epoch time: 53.86s

Epoch [20/30] α=0.10 T=0.0489 subset=2736
TrainAcc=0.89 TrainLoss=0.3208 | TestAcc=0.8436 TestLoss=0.4728

[SA] Metropolis: δ=+0.00 r=0.93 boltz=1.00
[SA] ACCEPT δ=+0.00 α→1.10
Epoch time: 46.12s

Epoch [21/30] α=1.10 T=0.0418 subset=4751
TrainAcc=0.90 TrainLoss=0.2884 | TestAcc=0.8449 TestLoss=0.4749

[SA] ACCEPT δ=+0.01 α→2.10
Epoch time: 53.10s

Epoch [22/30] α=2.10 T=0.0356 subset=6431
TrainAcc=0.90 TrainLoss=0.2915 | TestAcc=0.8533 TestLoss=0.4647

[SA] Metropolis: δ=+0.00 r=0.80 boltz=1.00
[SA] ACCEPT δ=+0.00 α→1.10
Epoch time: 57.73s

Epoch [23/30] α=1.10 T=0.0304 subset=4356
TrainAcc=0.89 TrainLoss=0.3047 | TestAcc=0.8462 TestLoss=0.4826

[SA] Metropolis: δ=-0.01 r=0.23 boltz=0.72
[SA] ACCEPT δ=-0.01 α→0.10
Epoch time: 52.11s

Epoch [24/30] α=0.10 T=0.0259 subset=2282
TrainAcc=0.91 TrainLoss=0.2689 | TestAcc=0.8495 TestLoss=0.4639

[SA] ACCEPT δ=+0.02 α→1.10
Epoch time: 45.35s

Epoch [25/30] α=1.10 T=0.0221 subset=4022
TrainAcc=0.89 TrainLoss=0.3076 | TestAcc=0.8456 TestLoss=0.4775

[SA] Metropolis: δ=-0.02 r=0.50 boltz=0.40
[SA DEBUG] REJECT → intense spike: -3.38 new α=0.10
Epoch time: 50.32s

Epoch [26/30] α=0.10 T=0.0189 subset=2107
TrainAcc=0.90 TrainLoss=0.2887 | TestAcc=0.8465 TestLoss=0.4843

[SA] ACCEPT δ=+0.01 α→0.10
Epoch time: 44.13s

Epoch [27/30] α=0.10 T=0.0161 subset=2029
TrainAcc=0.91 TrainLoss=0.2558 | TestAcc=0.8501 TestLoss=0.4729

[SA] ACCEPT δ=+0.01 α→0.10
Epoch time: 44.59s

Epoch [28/30] α=0.10 T=0.0137 subset=1957
TrainAcc=0.90 TrainLoss=0.3005 | TestAcc=0.8427 TestLoss=0.4937

[SA] Metropolis: δ=-0.01 r=0.01 boltz=0.48
[SA] ACCEPT δ=-0.01 α→0.10
Epoch time: 43.63s

Epoch [29/30] α=0.10 T=0.0117 subset=1890
TrainAcc=0.88 TrainLoss=0.3697 | TestAcc=0.8369 TestLoss=0.5014

[SA] Metropolis: δ=-0.02 r=0.73 boltz=0.18
[SA DEBUG] REJECT → intense spike: -1.18 new α=0.10
Epoch time: 44.07s

Epoch [30/30] α=0.10 T=0.0100 subset=50000
TrainAcc=0.90 TrainLoss=0.3036 | TestAcc=0.8561 TestLoss=0.4428

Epoch time: 195.68s
Training Time: 2311.79 seconds
Memory Consumption: 5685.28 MB

Total backprop samples: 372575
Effective epochs (float): 7.452
Info: Created directory 'cifar10_results_30epochs' for JSON data file.
Info: JSON file 'cifar10_results_30epochs/efficientnet_b0' not found. Initializing fresh data.
Info: Data for 'efficientnet_b0_0.3_SA_simple' saved to 'cifar10_results_30epochs/efficientnet_b0'.
Info: Plot saved to 'cifar10_results_30epochs/efficientnet_b0'.
Info: Test data JSON file 'cifar10_results_30epochs/efficientnet_b0_test' not found. Initializing fresh data.
Info: Test data for 'efficientnet_b0_0.3_SA_simple' saved to 'cifar10_results_30epochs/efficientnet_b0_test'.
Info: Test accuracy plot saved to 'cifar10_results_30epochs/efficientnet_b0_test'.
Info: Epoch samples plot saved to 'cifar10_results_30epochs/efficientnet_b0_test_epochs_00.png'.
Number of steps: 372575
Effective Epochs: 7
(venv) ➜ energy_eff_training

```
"""
saRevisionSimple.py
Simulated Annealing–driven dynamic dropout scheduler (Simple)
------------------------------------------------------------
Implements function-based SMRD approximation + per-epoch dataset reduction
with simulated annealing tuning of α.

SA mechanics:
- alpha controls the SRD decay curve steepness (higher alpha = more aggressiv
e dropout).
- Each epoch a step of ±SA_ALPHA_STEP is proposed at random.
- ACCEPT  → α takes the proposed value.
- REJECT  → intense downward spike: α -= abs(N(0, step*3)) which forces α low
er
             (less aggressive dropout, more samples) as an exploration burst.
             The asymmetric force means a reject always loosens the schedule,
             promoting recovery before the next cooling attempt.

Fairness:
- delta, r, and Boltzmann probability are all rounded to 2 decimal places
  before comparison to avoid floating-point asymmetries in the accept/reject
  decision.
```

```
Gradient-scale fix (considered):
- PDD concern: if dropout is applied as in-batch masking (batch[mask]), the
  retained tensor is smaller, so reduction='mean' averages over fewer samples
  → each kept sample contributes more to the gradient → hidden effective-LR
  increase as training progresses.
- This file uses SubsetRandomSampler to select K_t samples BEFORE building th
e
  DataLoader, so every batch that IS processed is a full batch_size batch.
  reduction='mean' is therefore consistent across all epochs — no hidden scal
e
  change. The concern does NOT apply here.
- If you switch to in-batch masking, change to:
    loss = criterion_sum(outputs, labels_retained) / original_batch_size

LR schedule fix (NOT applied — intentional for simplicity):
- Concern: StepLR(step_size=1) decays LR once per epoch. Later PDD epochs
  have fewer batches (smaller K_t) → LR decays after fewer optimizer steps
  than in baseline → hidden faster LR decay that could confound results.
- For SA we accept this confound to keep the design minimal. If you need a
  clean ablation, replace with step-based decay (see gaRevisionFinal.py).
"""

import math
import time
import numpy as np
from dataclasses import dataclass
from typing import Dict, Callable
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import StepLR
from torch.utils.data import DataLoader, SubsetRandomSampler
from tqdm import tqdm

from utils import (
    log_memory,
    plot_accuracy_time_multi,
    plot_accuracy_time_multi_test,
)
#global hyperparameters for SA (edit here only)
SA_DECAY_KIND = "inverse_linear"   # "log", "inverse_linear", "power_law", "e
xponential", "sigmoid_complement"
SA_ALPHA_INIT = 2.0
SA_ALPHA_STEP = 1.0
```

```
SA_ALPHA_MIN = 0.10
SA_ALPHA_MAX = 20.0
SA_T0 = 1.0
SA_TFINAL = 1e-2
SA_COOLING = "geometric"# "linear" or "geometric"
REVISION_FINAL_EPOCH = True# triggers final epoch(s) using full dataset
REVISION_EPOCHS = 1
SA_LR = 3e-4
SA_GAMMA = 0.98
NUM_WORKERS = 4


#dataclass for config
@dataclass
class SAConfig:
    seed:           int   = 42
    decay_kind:     str   = SA_DECAY_KIND
    alpha_init:     float = SA_ALPHA_INIT
    alpha_step:     float = SA_ALPHA_STEP
    T0:             float = SA_T0
    Tfinal:         float = SA_TFINAL
    cooling:        str   = SA_COOLING
    revision_epochs: int  = REVISION_EPOCHS
    revision_flag:  bool  = REVISION_FINAL_EPOCH


#decay functions (SRD curve family)
def power_law(epoch, T, data_size, alpha):
    x = np.arange(1, T + 1)
    y = 1 / (x ** alpha)
    y_scaled = (y / y.max()) * data_size
    return int(y_scaled[epoch - 1])

def exponential(epoch, T, data_size, alpha):
    x = np.arange(1, T + 1)
    y = np.exp(-alpha * x)
    y_scaled = (y / y.max()) * data_size
    return int(y_scaled[epoch - 1])

def logarithmic(epoch, T, data_size, alpha):
    a = max(alpha, 1.0001)
    x = np.arange(1, T + 1)
    y = 1 / np.log(x + a)
    y_scaled = (y / y.max()) * data_size
```

```
    return int(y_scaled[epoch - 1])

def inverse_linear(epoch, T, data_size, alpha):
    x = np.arange(1, T + 1)
    y = 1 / (x + alpha)
    y_scaled = (y / y.max()) * data_size
    return int(y_scaled[epoch - 1])

def sigmoid_complement(epoch, T, data_size, alpha):
    x = np.arange(1, T + 1)
    y = 1 - 1 / (1 + np.exp(-alpha * x))
    y_scaled = (y / y.max()) * data_size
    return int(y_scaled[epoch - 1])

DECAY_FUNCS: Dict[str, Callable] = {
    "power_law":          power_law,
    "exponential":        exponential,
    "log":                logarithmic,
    "inverse_linear":     inverse_linear,
    "sigmoid_complement": sigmoid_complement,
}


#SimAnnealDropoutSimple class
class SimAnnealDropoutSimple:
    def __init__(self, model_name, model, train_loader, test_loader, device,
                 epochs, save_path, threshold):
        self.model_name   = model_name
        self.model        = model
        self.train_loader = train_loader
        self.test_loader  = test_loader
        self.device       = device
        self.epochs       = epochs
        self.save_path    = save_path
        self.dataset      = train_loader.dataset
        self.data_size    = len(self.dataset)
        self.batch_size   = train_loader.batch_size
        self.num_workers  = NUM_WORKERS

    def _build_subset_loader(self, subset_size, seed=None):
        N = len(self.dataset)
        subset_size = min(max(1, subset_size), N)
        g = torch.Generator()
        g.manual_seed(seed if seed is not None else torch.randint(0, 2**31 -
```

```
1, (1,)).item())
        idx = torch.randperm(N, generator=g)[:subset_size]
        sampler = SubsetRandomSampler(idx.tolist())
        return DataLoader(
            self.dataset, batch_size=self.batch_size, sampler=sampler,
            num_workers=self.num_workers,
            pin_memory=(self.device.type == "cuda"),
        )

    @torch.no_grad()
    def _eval(self):
        self.model.eval()
        total, correct, loss_sum = 0, 0, 0.0
        criterion = nn.CrossEntropyLoss()
        for inputs, labels in self.test_loader:
            inputs, labels = inputs.to(self.device), labels.to(self.device)
            outputs  = self.model(inputs)
            loss_sum += criterion(outputs, labels).item()
            correct  += (outputs.argmax(dim=1) == labels).sum().item()
            total    += labels.size(0)
        return correct / total, loss_sum / len(self.test_loader)

    def train_with_sa(self, cfg: SAConfig):
        decay_fn = DECAY_FUNCS[cfg.decay_kind]
        alpha = cfg.alpha_init
        T = cfg.T0
        optimizer = optim.AdamW(self.model.parameters(), lr=SA_LR)

        # LR SCHEDULE: epoch-based (step_size=1 → decay once per epoch).
        # Confound acknowledged: as K_t shrinks, each epoch has fewer batche
s,
        # so LR decays after fewer optimizer steps than baseline.
        # Accepted here for SA simplicity. See gaRevisionFinal.py for the
        # step-based fix that removes this confound.
        scheduler = StepLR(optimizer, step_size=1, gamma=SA_GAMMA)

        # GRADIENT SCALE: reduction='mean' is safe here because SubsetSampler
        # fills every batch to batch_size before the DataLoader is called.
        # No in-batch masking → no hidden per-sample gradient scaling.
        criterion = nn.CrossEntropyLoss()

        rng = np.random.default_rng(cfg.seed)
        total_bp = 0
        prev_acc = 0.00   # 2dp baseline
```

```
        epoch_losses, epoch_accuracies           = [], []
        epoch_test_accuracies, epoch_test_losses = [], []
        samples_used_per_epoch, time_per_epoch   = [], []

        start_time = time.time()

        for epoch in range(1, self.epochs + 1):
            epoch_start = time.time()

            # ---- dataset size for this epoch ----
            is_revision = cfg.revision_flag and (epoch > self.epochs - cfg.re
vision_epochs)
            subset_size = self.data_size if is_revision else \
                          decay_fn(epoch, self.epochs, self.data_size, alpha)
            subset_size = max(subset_size, self.batch_size)
            loader      = self._build_subset_loader(subset_size, seed=cfg.see
d + epoch)

            print(f"\nEpoch [{epoch}/{self.epochs}] α={alpha:.2f} T={T:.4f} s
ubset={subset_size}")
            self.model.train()
            running_loss, correct, total = 0.0, 0, 0

            for inputs, labels in tqdm(loader, desc="Training", leave=False):
                inputs, labels = inputs.to(self.device), labels.to(self.devic
e)
                optimizer.zero_grad()
                outputs = self.model(inputs)
                loss    = criterion(outputs, labels)
                loss.backward()
                optimizer.step()
                running_loss += loss.item()
                correct      += (outputs.argmax(1) == labels).sum().item()
                total        += labels.size(0)
                total_bp     += labels.size(0)

            # round to 2dp so all SA comparisons are on equal footing
            train_acc  = round(correct / total, 2)
            epoch_loss = running_loss / len(loader)
            samples_used_per_epoch.append(subset_size)
            epoch_accuracies.append(train_acc)
            epoch_losses.append(epoch_loss)
```

```
            test_acc, val_loss = self._eval()
            scheduler.step(val_loss)
            epoch_test_accuracies.append(test_acc)
            epoch_test_losses.append(val_loss)

            print(f"TrainAcc={train_acc:.2f}  TrainLoss={epoch_loss:.4f} | "
                  f"TestAcc={test_acc:.4f}  TestLoss={val_loss:.4f}")

            # ---- SA update ----
            if not is_revision and epoch < self.epochs:
                delta   = round(train_acc - prev_acc, 2)# 2dp delta
                a_prime = round(float(np.clip(
                    alpha + rng.choice([-1, 1]) * cfg.alpha_step,
                    SA_ALPHA_MIN, SA_ALPHA_MAX)), 2)

                accept = False
                reason = ""
                r, boltz = "-", "-"# shown as dash when Metropolis not reache
d

                if delta > 0.00:
                    # direct improvement — no Metropolis check needed
                    accept = True
                    reason = "ACCEPT (improvement)"
                else:
                    # Metropolis criterion — all at 2dp for fair comparison
                    r     = round(float(rng.random()), 2)
                    boltz = round(math.exp(delta / max(T, 1e-12)), 2)
                    if r < boltz:
                        accept = True
                        reason = "ACCEPT (Metropolis)"
                    else:
                        reason = "REJECT → mutate"

                print(f"[SA] δ={delta:+.2f}  r={r}  boltz={boltz}  T={T:.4f}
→ {reason}")

                if accept:
                    alpha = a_prime
                    print(f"[SA]   proposed α={alpha:.2f} accepted")
                else:
                    # REJECT: intense asymmetric spike — always lowers alpha
                    # (lower alpha = less aggressive dropout = more samples n
ext epoch)
```

```
                    noise    = float(rng.normal(0, cfg.alpha_step * 3.0))
                    alpha_old = alpha
                    alpha    = round(float(np.clip(alpha - abs(noise), SA_ALP
HA_MIN, SA_ALPHA_MAX)), 2)
                    print(f"[SA]   spike: α {alpha_old:.2f} → {alpha:.2f}  "
                          f"(−{abs(noise):.2f}, more samples next epoch)")

                # ---- cooling schedule ----
                if cfg.cooling == "geometric":
                    ratio = (cfg.Tfinal / cfg.T0) ** (1.0 / max(self.epochs -
1, 1))
                    T     = max(T * ratio, cfg.Tfinal)
                else:
                    T     = max(T - (cfg.T0 - cfg.Tfinal) / self.epochs, cfg.
Tfinal)

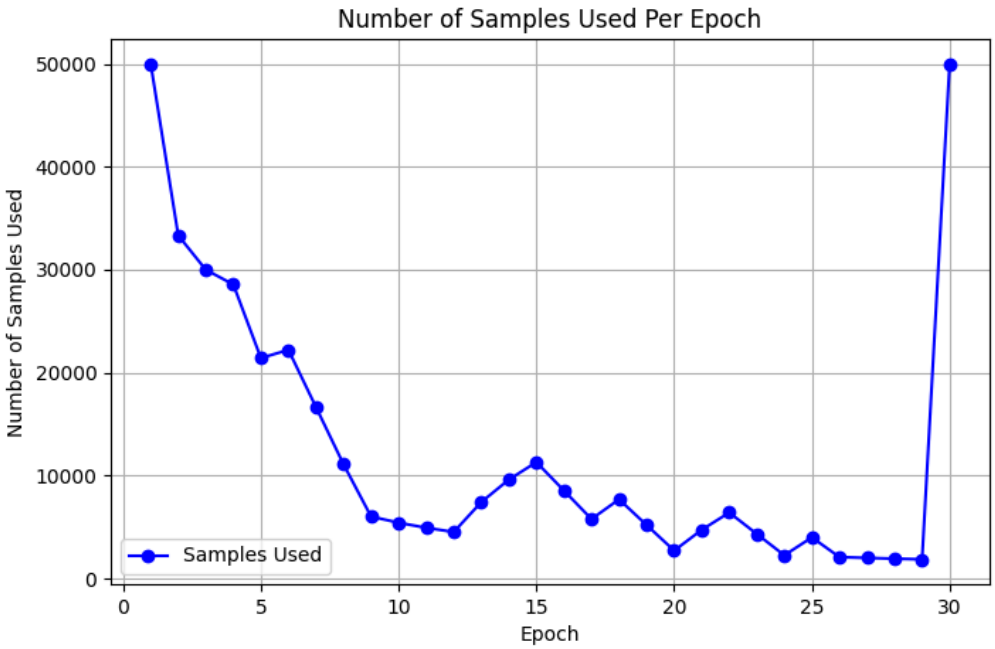
            prev_acc   = train_acc
            epoch_time = time.time() - epoch_start
            time_per_epoch.append(epoch_time)
            print(f"Epoch time: {epoch_time:.2f}s")

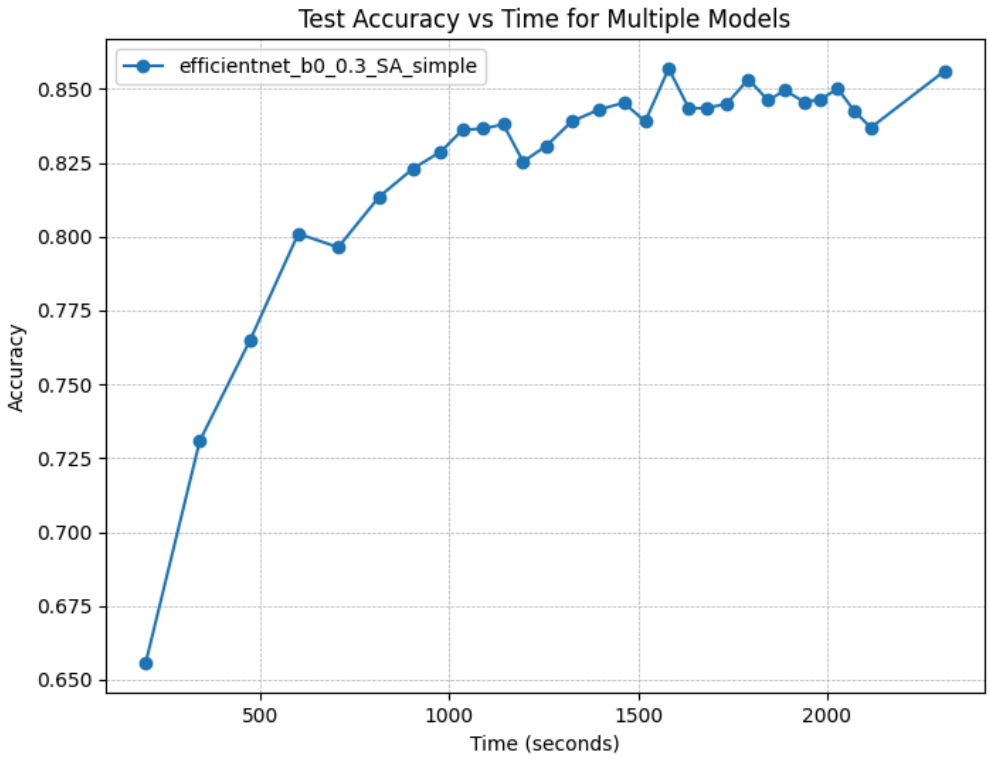
        end_time = time.time()
        log_memory(start_time, end_time)

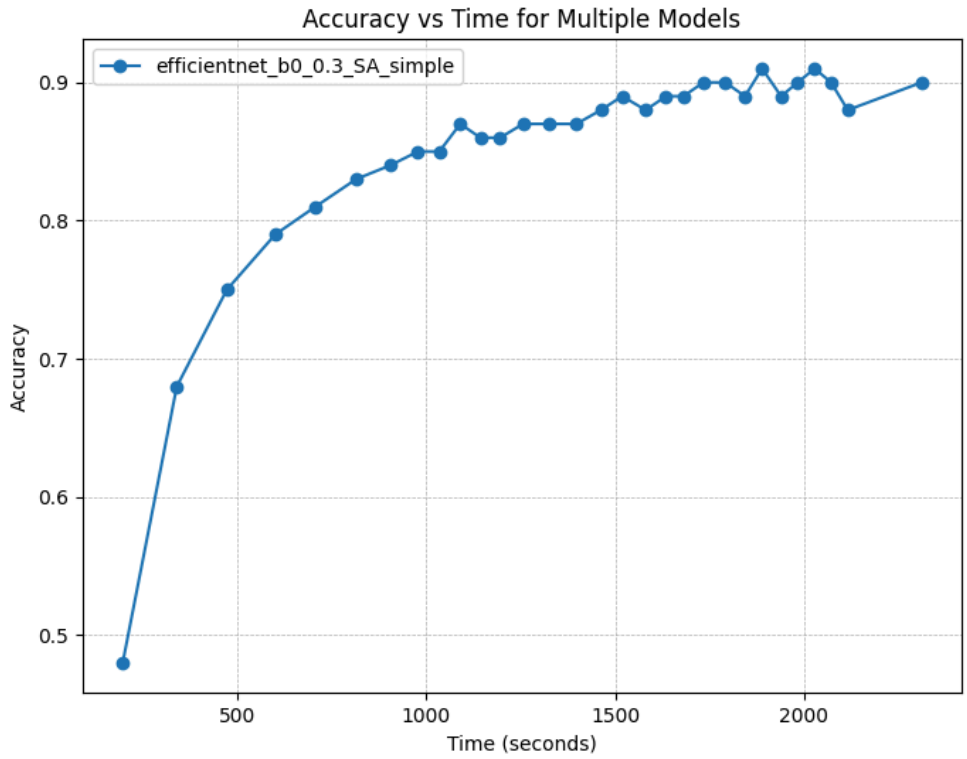
        eff_epochs = total_bp / self.data_size
        print(f"\nTotal backprop samples: {total_bp}")
        print(f"Effective epochs (float): {eff_epochs:.3f}")

        plot_accuracy_time_multi(
            model_name=self.model_name + "_SA_simple",
            accuracy=epoch_accuracies,
            time_per_epoch=time_per_epoch,
            save_path=self.save_path,
            data_file=self.save_path,
        )
        plot_accuracy_time_multi_test(
            model_name=self.model_name + "_SA_simple",
            accuracy=epoch_test_accuracies,
            time_per_epoch=time_per_epoch,
            samples_per_epoch=samples_used_per_epoch,
            threshold=0.0,
            save_path=self.save_path,
            data_file=self.save_path,
        )
```

```
        return self.model, int(total_bp)
```




Test Accuracy vs Time for Multiple Models
efficientnet_b0_0.3_SA_simple
Accuracy
Time (seconds)
0.650
0.675
0.700
0.725
0.750
0.775
0.800
0.825
0.850
500
1000
1500
2000



train_with_sa 30 epochs efficientnet b0

(venv) ➜ energy_eff_training python ./main.py \
--model efficientnet_b0 \
--mode train_with_sa \
--epoch 30 \
--save_path cifar10_results_30epochs/efficientnet_b0 \
--dataset cifar10 \
--batch_size 32 \
--start_revision 0 \
--task classification \
--threshold 0.3 \
--seed 123
Using device: mps
Apple Silicon GPU (MPS) enabled.
/Users/amar/Developer/energy_eff_training/venv/lib/python3.13/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)

entry = pickle.load(f, encoding="latin1")
[DATA] Epoch 1: Jaccard(prev_subset) = N/A (first epoch)

Epoch [1/30] T_epoch=1.0000 subset=50000
Training: 0%|
| 0/1563 [00:00<?, ?it/s]/Users/amar/Developer/energy_eff_training/venv/lib/python3.13/site-packages/torch/utils/data/dataloader.py:1118: UserWarning: 'pin_memory' argument is set as true but not supported on MPS now, device pinned memory won't be used.
super().**init**(loader)
TrainAcc=0.4769 TestAcc=0.6566

[SA][ACCEPT:threshold] epoch=1 delta=+0.4769 T: 1.0000 → 0.5607
Epoch time: 195.9110s
[DATA] Epoch 2: Jaccard(prev_subset) = 0.5607 (|curr|=28035, |prev|=50000, |∩|=28035)

Epoch [2/30] T_epoch=0.5607 subset=28035
TrainAcc=0.6725 TestAcc=0.7355

[SA][ACCEPT:threshold] epoch=2 delta=+0.1956 T: 0.5607 → 0.4049
Epoch time: 126.2982s
[DATA] Epoch 3: Jaccard(prev_subset) = 0.3086 (|curr|=20245, |prev|=28035, |∩|=11387)

Epoch [3/30] T_epoch=0.4049 subset=20245
TrainAcc=0.7354 TestAcc=0.7612

[SA][ACCEPT:threshold] epoch=3 delta=+0.0629 T: 0.4049 → 0.3224
Epoch time: 101.5877s
[DATA] Epoch 4: Jaccard(prev_subset) = 0.2163 (|curr|=16120, |prev|=20245, |∩|=6466)

Epoch [4/30] T_epoch=0.3224 subset=16120
TrainAcc=0.7638 TestAcc=0.7797

[SA][ACCEPT:threshold] epoch=4 delta=+0.0284 T: 0.3224 → 0.2702
Epoch time: 88.5097s
[DATA] Epoch 5: Jaccard(prev_subset) = 0.1743 (|curr|=13510, |prev|=16120, |∩|=4399)

Epoch [5/30] T_epoch=0.2702 subset=13510
TrainAcc=0.7871 TestAcc=0.7876

[SA][ACCEPT:threshold] epoch=5 delta=+0.0233 T: 0.2702 → 0.2337
Epoch time: 80.4936s
[DATA] Epoch 6: Jaccard(prev_subset) = 0.1455 (|curr|=11685, |prev|=13510, |∩|=3201)

Epoch [6/30] T_epoch=0.2337 subset=11685
TrainAcc=0.8025 TestAcc=0.7965

[SA][ACCEPT:threshold] epoch=6 delta=+0.0154 T: 0.2337 → 0.2065
Epoch time: 231.1767s
[DATA] Epoch 7: Jaccard(prev_subset) = 0.1204 (|curr|=10325, |prev|=11685, |∩|=2365)

Epoch [7/30] T_epoch=0.2065 subset=10325
TrainAcc=0.8057 TestAcc=0.8035

[SA][DEBUG] epoch=7 delta=+0.0032, T=0.2065, r=0.6824, boltz_prob_raw=1.0395, accept_prob=1.0395, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=7 delta=+0.0032 T: 0.2065 → 0.1852
Epoch time: 120.2725s
[DATA] Epoch 8: Jaccard(prev_subset) = 0.1057 (|curr|=9260, |prev|=10325, |∩|=1873)

Epoch [8/30] T_epoch=0.1852 subset=9260
TrainAcc=0.8146 TestAcc=0.8090

[SA][ACCEPT:threshold] epoch=8 delta=+0.0089 T: 0.1852 → 0.1681
Epoch time: 67.1691s
[DATA] Epoch 9: Jaccard(prev_subset) = 0.0943 (|curr|=8405, |prev|=9260, |∩|=1522)

Epoch [9/30] T_epoch=0.1681 subset=8405
TrainAcc=0.8256 TestAcc=0.8107

[SA][ACCEPT:threshold] epoch=9 delta=+0.0110 T: 0.1681 → 0.1538
Epoch time: 64.0053s
[DATA] Epoch 10: Jaccard(prev_subset) = 0.0873 (|curr|=7690, |prev|=8405, |∩|=1292)

Epoch [10/30] T_epoch=0.1538 subset=7690
TrainAcc=0.8336 TestAcc=0.8188

[SA][ACCEPT:threshold] epoch=10 delta=+0.0080 T: 0.1538 → 0.1418
Epoch time: 62.4926s
[DATA] Epoch 11: Jaccard(prev_subset) = 0.0809 (|curr|=7090, |prev|=7690, |∩|=1106)

Epoch [11/30] T_epoch=0.1418 subset=7090
TrainAcc=0.8382 TestAcc=0.8193

[SA][DEBUG] epoch=11 delta=+0.0046, T=0.1418, r=0.0538, boltz_prob_raw=1.0845, accept_prob=1.0845, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=11 delta=+0.0046 T: 0.1418 → 0.1314
Epoch time: 60.6662s
[DATA] Epoch 12: Jaccard(prev_subset) = 0.0730 (|curr|=6570, |prev|=7090, |∩|=929)

Epoch [12/30] T_epoch=0.1314 subset=6570
TrainAcc=0.8387 TestAcc=0.8208

[SA][DEBUG] epoch=12 delta=+0.0005, T=0.1314, r=0.2204, boltz_prob_raw=1.0096, accept_prob=1.0096, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=12 delta=+0.0005 T: 0.1314 → 0.1224
Epoch time: 58.2874s
[DATA] Epoch 13: Jaccard(prev_subset) = 0.0664 (|curr|=6120, |prev|=6570, |∩|=790)

Epoch [13/30] T_epoch=0.1224 subset=6120
TrainAcc=0.8471 TestAcc=0.8270

[SA][ACCEPT:threshold] epoch=13 delta=+0.0084 T: 0.1224 → 0.1144
Epoch time: 57.6014s
[DATA] Epoch 14: Jaccard(prev_subset) = 0.0612 (|curr|=5720, |prev|=6120, |∩|=683)

Epoch [14/30] T_epoch=0.1144 subset=5720
TrainAcc=0.8512 TestAcc=0.8251

[SA][DEBUG] epoch=14 delta=+0.0041, T=0.1144, r=0.1844, boltz_prob_raw=1.0937, accept_prob=1.0937, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=14 delta=+0.0041 T: 0.1144 → 0.1073
Epoch time: 55.4466s
[DATA] Epoch 15: Jaccard(prev_subset) = 0.0600 (|curr|=5365, |prev|=5720, |∩|=627)

Epoch [15/30] T_epoch=0.1073 subset=5365
TrainAcc=0.8595 TestAcc=0.8263

[SA][ACCEPT:threshold] epoch=15 delta=+0.0083 T: 0.1073 → 0.1009
Epoch time: 54.3468s
[DATA] Epoch 16: Jaccard(prev_subset) = 0.0555 (|curr|=5045, |prev|=5365, |∩|=547)

Epoch [16/30] T_epoch=0.1009 subset=5045
TrainAcc=0.8476 TestAcc=0.8242

[SA][DEBUG] epoch=16 delta=-0.0119, T=0.1009, r=0.1759, boltz_prob_raw=0.7446, accept_prob=0.7446, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=16 delta=-0.0119 T: 0.1009 → 0.0952
Epoch time: 53.3400s
[DATA] Epoch 17: Jaccard(prev_subset) = 0.0510 (|curr|=4760, |prev|=5045, |∩|=476)

Epoch [17/30] T_epoch=0.0952 subset=4760
TrainAcc=0.8622 TestAcc=0.8268

[SA][ACCEPT:threshold] epoch=17 delta=+0.0146 T: 0.0952 → 0.0899
Epoch time: 52.4998s
[DATA] Epoch 18: Jaccard(prev_subset) = 0.0483 (|curr|=4495, |prev|=4760, |∩|=426)

Epoch [18/30] T_epoch=0.0899 subset=4495
TrainAcc=0.8603 TestAcc=0.8311

[SA][DEBUG] epoch=18 delta=-0.0019, T=0.0899, r=0.8121, boltz_prob_raw=0.9485, accept_prob=0.9485, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=18 delta=-0.0019 T: 0.0899 → 0.0852
Epoch time: 52.3066s
[DATA] Epoch 19: Jaccard(prev_subset) = 0.0433 (|curr|=4260, |prev|=4495, |∩|=363)

Epoch [19/30] T_epoch=0.0852 subset=4260
TrainAcc=0.8667 TestAcc=0.8260

[SA][ACCEPT:threshold] epoch=19 delta=+0.0064 T: 0.0852 → 0.0807
Epoch time: 51.4087s
[DATA] Epoch 20: Jaccard(prev_subset) = 0.0431 (|curr|=4035, |prev|=4260, |∩|=343)

Epoch [20/30] T_epoch=0.0807 subset=4035
TrainAcc=0.8570 TestAcc=0.8212

[SA][DEBUG] epoch=20 delta=-0.0097, T=0.0807, r=0.9233, boltz_prob_raw=0.7405, accept_prob=0.7405, scalar=2.5000

[SA][REHEAT] epoch=20 delta=-0.0097 T: 0.0807 → 0.2017
Epoch time: 50.2019s
[DATA] Epoch 21: Jaccard(prev_subset) = 0.0624 (|curr|=10085, |prev|=4035, |∩|=829)

Epoch [21/30] T_epoch=0.2017 subset=10085
TrainAcc=0.8666 TestAcc=0.8312

[SA][ACCEPT:threshold] epoch=21 delta=+0.0096 T: 0.2017 → 0.0729
Epoch time: 69.2802s
[DATA] Epoch 22: Jaccard(prev_subset) = 0.0549 (|curr|=3645, |prev|=10085, |∩|=715)

Epoch [22/30] T_epoch=0.0729 subset=3645
TrainAcc=0.8757 TestAcc=0.8316

[SA][ACCEPT:threshold] epoch=22 delta=+0.0091 T: 0.0729 → 0.0693
Epoch time: 49.4249s
[DATA] Epoch 23: Jaccard(prev_subset) = 0.0339 (|curr|=3465, |prev|=3645, |∩|=233)

Epoch [23/30] T_epoch=0.0693 subset=3465
TrainAcc=0.8802 TestAcc=0.8461

[SA][DEBUG] epoch=23 delta=+0.0045, T=0.0693, r=0.2766, boltz_prob_raw=1.1763, accept_prob=1.1763, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=23 delta=+0.0045 T: 0.0693 → 0.0661
Epoch time: 48.8515s
[DATA] Epoch 24: Jaccard(prev_subset) = 0.0363 (|curr|=3305, |prev|=3465, |∩|=237)

Epoch [24/30] T_epoch=0.0661 subset=3305
TrainAcc=0.8817 TestAcc=0.8431

[SA][DEBUG] epoch=24 delta=+0.0015, T=0.0661, r=0.8198, boltz_prob_raw=1.0584, accept_prob=1.0584, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=24 delta=+0.0015 T: 0.0661 → 0.0630
Epoch time: 47.8500s
[DATA] Epoch 25: Jaccard(prev_subset) = 0.0336 (|curr|=3150, |prev|=3305, |∩|=210)

Epoch [25/30] T_epoch=0.0630 subset=3150
TrainAcc=0.8727 TestAcc=0.8353

[SA][DEBUG] epoch=25 delta=-0.0090, T=0.0630, r=0.8899, boltz_prob_raw=0.6997, accept_prob=0.6997, scalar=2.5000
[SA][REHEAT] epoch=25 delta=-0.0090 T: 0.0630 → 0.1575
Epoch time: 47.8427s
[DATA] Epoch 26: Jaccard(prev_subset) = 0.0481 (|curr|=7875, |prev|=3150, |∩|=506)

Epoch [26/30] T_epoch=0.1575 subset=7875
TrainAcc=0.8724 TestAcc=0.8348

[SA][DEBUG] epoch=26 delta=-0.0003, T=0.1575, r=0.5130, boltz_prob_raw=0.9952, accept_prob=0.9952, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=26 delta=-0.0003 T: 0.1575 → 0.0573

Epoch time: 62.2466s
[DATA] Epoch 27: Jaccard(prev_subset) = 0.0467 (|curr|=2865, |prev|=7875, |∩|=479)

Epoch [27/30] T_epoch=0.0573 subset=2865
TrainAcc=0.8911 TestAcc=0.8427

[SA][ACCEPT:threshold] epoch=27 delta=+0.0187 T: 0.0573 → 0.0548
Epoch time: 46.9759s
[DATA] Epoch 28: Jaccard(prev_subset) = 0.0277 (|curr|=2740, |prev|=2865, |∩|=151)

Epoch [28/30] T_epoch=0.0548 subset=2740
TrainAcc=0.8766 TestAcc=0.8475

[SA][DEBUG] epoch=28 delta=-0.0145, T=0.0548, r=0.2450, boltz_prob_raw=0.5161,
accept_prob=0.5161, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=28 delta=-0.0145 T: 0.0548 → 0.0523
Epoch time: 46.5840s
[DATA] Epoch 29: Jaccard(prev_subset) = 0.0239 (|curr|=2615, |prev|=2740, |∩|=125)

Epoch [29/30] T_epoch=0.0523 subset=2615
TrainAcc=0.8822 TestAcc=0.8380

[SA][ACCEPT:threshold] epoch=29 delta=+0.0056 T: 0.0523 → 0.0500
Epoch time: 46.2579s
[DATA] Epoch 30: Jaccard(prev_subset) = 0.0523 (|curr|=50000, |prev|=2615, |∩|=2615)

Epoch [30/30] T_epoch=1.0000 subset=50000
TrainAcc=0.8805 TestAcc=0.8528

Epoch time: 195.9987s
Training Time: 2345.33 seconds
Memory Consumption: 5562.84 MB

Total backprop samples: 318480
Effective epochs (float): 6.3696
Info: Data for 'efficientnet_b0_0.3_SA_v3_scalar' saved to 'cifar10_results_30epochs/efficientnet_b0'.
Info: Plot saved to 'cifar10_results_30epochs/efficientnet_b0'.
Info: Test data for 'efficientnet_b0_0.3_SA_v3_scalar' saved to
'cifar10_results_30epochs/efficientnet_b0_test'.
Info: Test accuracy plot saved to 'cifar10_results_30epochs/efficientnet_b0_test'.
Info: Epoch samples plot saved to 'cifar10_results_30epochs/efficientnet_b0_test_epochs_00.png'.
Number of steps: 318480
Effective Epochs: 6
(venv) ➜ energy_eff_training

```
"""
SARevision.py
Simulated Annealing–driven dynamic dropout scheduler
- No explicit alpha parameter.
- Each epoch uses a random subset of the *full* dataset (no replacement withi
```

```
n
  epoch, but samples may reappear across epochs).
- Temperature T directly controls the fraction of data kept.
- Cooling schedule (linear, geometric, or one of the analytic curves below)
  proposes a lower T; SA acceptance can either:
- accept the cooler T (smaller subset), or
- reject and "reheat" by increasing T (larger subset).
- This variant controls reheating frequency by scaling the Boltzmann exponen
t.
"""
import math
import time
from dataclasses import dataclass
from typing import Dict, Callable
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import StepLR
from torch.utils.data import DataLoader, SubsetRandomSampler
from tqdm import tqdm
import numpy as np

# plotting utils (same as in TrainRevision)
from utils import (
    log_memory,
    plot_accuracy_time_multi,
    plot_accuracy_time_multi_test,
)

#global hyperparameters for SA (edit here only)
# SA_COOLING can be:
# "linear", "geometric",
# or one of: "power_law", "exponential", "log",
# "inverse_linear", "sigmoid_complement"
SA_COOLING = "log"
SA_T0 = 1.0000 # initial keep-fraction (use 100% of dataset)
SA_TFINAL = 0.0500 # final keep-fraction (e.g. 5% of dataset)
SA_DELTA_THRESHOLD = 0.0050 # accuracy improvement threshold to auto-accept c
ooling
SA_REHEAT_MULT = 2.5000 # how strongly to reheat (T <- min(T0, T * SA_REHEAT_
MULT))

#Reheating control: Boltzmann scalar
SA_BOLTZ_SCALAR = 2.5000 #>1 => more acceptance (less reheating), <1 => more
```

```
reheating
REVISION_FINAL_EPOCH = True # final revision epochs with full dataset
REVISION_EPOCHS = 1 # number of revision epochs
SA_LR = 3e-4
SA_GAMMA = 0.9800 # LR scheduler gamma (StepLR), NOT SA temperature
NUM_WORKERS = 4

@dataclass
class SAConfig:
    seed: int = 42
    T0: float = SA_T0
    Tfinal: float = SA_TFINAL
    cooling: str = SA_COOLING
    revision_epochs: int = REVISION_EPOCHS
    revision_flag: bool = REVISION_FINAL_EPOCH
    delta_threshold: float = SA_DELTA_THRESHOLD
    boltz_scalar: float = SA_BOLTZ_SCALAR


#analytic cooling-shape functions
def power_law(epoch: int, total_epochs: int, data_size: float, alpha: float)
-> float:
    x = np.arange(1, total_epochs + 1)
    y = 1.0 / (x ** alpha)
    y_scaled = (y / y.max()) * data_size
    return float(y_scaled[epoch - 1])


def exponential(epoch: int, total_epochs: int, data_size: float, alpha: floa
t) -> float:
    x = np.arange(1, total_epochs + 1)
    y = np.exp(-alpha * x)
    y_scaled = (y / y.max()) * data_size
    return float(y_scaled[epoch - 1])


def logarithmic(epoch: int, total_epochs: int, data_size: float, alpha: floa
t) -> float:
    a = max(alpha, 1.0001)
    x = np.arange(1, total_epochs + 1)
    y = 1.0 / np.log(x + a)
    y_scaled = (y / y.max()) * data_size
    return float(y_scaled[epoch - 1])
```

```
def inverse_linear(epoch: int, total_epochs: int, data_size: float, alpha: fl
oat) -> float:
    x = np.arange(1, total_epochs + 1)
    y = 1.0 / (x + alpha)
    y_scaled = (y / y.max()) * data_size
    return float(y_scaled[epoch - 1])


def sigmoid_complement(epoch: int, total_epochs: int, data_size: float, alph
a: float) -> float:
    x = np.arange(1, total_epochs + 1)
    y = 1.0 - 1.0 / (1.0 + np.exp(-alpha * x))
    y_scaled = (y / y.max()) * data_size
    return float(y_scaled[epoch - 1])


DECAY_FUNCS: Dict[str, Callable] = {
    "power_law": power_law,
    "exponential": exponential,
    "log": logarithmic,
    "inverse_linear": inverse_linear,
    "sigmoid_complement": sigmoid_complement,
}

class SimAnnealDropout:
    def __init__(self, model_name, model, train_loader, test_loader, device,
epochs, save_path, threshold):
        self.model_name = model_name
        self.model = model
        self.train_loader = train_loader
        self.test_loader = test_loader
        self.device = device
        self.epochs = epochs
        self.save_path = save_path
        self.dataset = train_loader.dataset
        self.data_size = len(self.dataset)
        self.batch_size = train_loader.batch_size
        self.num_workers = NUM_WORKERS

    def _build_subset_loader(self, subset_size, seed=None):
        """
        Version (1) behaviour:
        - Sample a random subset from the *full* dataset every epoch.
```

```
        - Without replacement *within* epoch (SubsetRandomSampler),
          but samples may reappear across epochs.
        Returns:
            loader, idx_list (indices used this epoch).
        """
        N = len(self.dataset)
        subset_size = min(max(1, subset_size), N)
        g = torch.Generator()
        if seed is not None:
            g.manual_seed(seed)
        else:
            g.manual_seed(torch.randint(0, 2**31 - 1, (1,)).item())
        idx = torch.randperm(N, generator=g)[:subset_size]
        idx_list = idx.tolist()
        sampler = SubsetRandomSampler(idx_list)
        loader = DataLoader(
            self.dataset,
            batch_size=self.batch_size,
            sampler=sampler,
            num_workers=self.num_workers,
            pin_memory=True,
        )
        return loader, idx_list

    @torch.no_grad()
    def _eval(self):
        self.model.eval()
        total, correct, loss_sum = 0, 0, 0.0
        criterion = nn.CrossEntropyLoss()
        for inputs, labels in self.test_loader:
            inputs, labels = inputs.to(self.device), labels.to(self.device)
            outputs = self.model(inputs)
            loss_sum += criterion(outputs, labels).item()
            correct += (outputs.argmax(dim=1) == labels).sum().item()
            total += labels.size(0)

        acc = correct / max(total, 1)
        loss = loss_sum / max(len(self.test_loader), 1)
        return round(acc, 4), round(loss, 4)

    def _build_base_temperature_schedule(self, cfg: SAConfig):
        """
        Helper to build the *baseline* temperature schedule T_base(e)
        from epoch index -> keep-fraction, using:
```

```
          - "linear"
          - "geometric"
          - or one of the analytic curves in DECAY_FUNCS.
        All schedules are scaled to lie exactly in [cfg.Tfinal, cfg.T0].
        """
        cooling = cfg.cooling
        E = self.epochs

        if cooling == "linear":
            def linear_T(e: int) -> float:
                if E <= 1:
                    return round(cfg.Tfinal, 4)
                frac = (e - 1) / max(E - 1, 1)
                val = cfg.T0 + frac * (cfg.Tfinal - cfg.T0)
                return round(val, 4)

            return linear_T

        if cooling == "geometric":
            if E <= 1:
                ratio = 1.0
            else:
                ratio = (cfg.Tfinal / cfg.T0) ** (1.0 / max(E - 1, 1))

            def geometric_T(e: int) -> float:
                val = cfg.T0 * (ratio ** (e - 1))
                val = max(val, cfg.Tfinal)
                return round(val, 4)

            return geometric_T

        if cooling not in DECAY_FUNCS:
            raise ValueError(f"Unknown cooling schedule '{cooling}'")

        func = DECAY_FUNCS[cooling]
        y_vals = []
        for e in range(1, E + 1):
            y_vals.append(func(e, E, data_size=1.0, alpha=1.0))
        y_vals = np.array(y_vals, dtype=float)
        y_min = float(y_vals.min())
        y_max = float(y_vals.max())

        if y_max == y_min:
            def flat_T(e: int) -> float:
```

```
                return round(cfg.Tfinal, 4)
            return flat_T

        def shaped_T(e: int) -> float:
            y = float(y_vals[e - 1])
            norm = (y - y_min) / (y_max - y_min)
            val = cfg.Tfinal + norm * (cfg.T0 - cfg.Tfinal)
            return round(val, 4)

        return shaped_T

    def train_with_sa(self, cfg: SAConfig):
        """
        Main training loop for Version 3 (boltz scalar).
        - Temperature T is interpreted as *keep-fraction* of the dataset.
        - Subset size per epoch = round(T * data_size), min = batch_size.
        - SA uses training accuracy improvement as the fitness of the current
          subset (for that epoch).
        - No explicit alpha; we only adjust T via cooling + reheating.
        """
        T_base = self._build_base_temperature_schedule(cfg)

        optimizer = optim.AdamW(self.model.parameters(), lr=SA_LR)

        # LR SCHEDULE FIX: step-based decay so that LR decays after the same
        # number of *optimizer steps* regardless of how large K_t is.
        # Epoch-based StepLR(step_size=1) would decay after fewer steps when
K_t
        # shrinks → hidden faster LR decay that confounds PDD vs baseline com
parison.
        # Fix: count gradient steps, decay once per equivalent full-dataset p
ass.
        steps_per_decay = math.ceil(self.data_size / self.batch_size)
        scheduler = StepLR(optimizer, step_size=steps_per_decay, gamma=SA_GAM
MA)

        # GRADIENT SCALE FIX: use reduction='sum' then divide by self.batch_s
ize
        # so that the mean is always over the *original* batch size, not the
        # actual number of samples in the (possibly smaller) last mini-batch.
        # Without this, the final mini-batch of an epoch contributes a higher
        # per-sample gradient weight than earlier full batches.
        criterion = nn.CrossEntropyLoss(reduction='sum')
```

```
        rng = np.random.default_rng(cfg.seed)
        total_bp = 0
        prev_acc = 0.0

        epoch_losses, epoch_accuracies = [], []
        epoch_test_accuracies, epoch_test_losses = [], []
        samples_used_per_epoch, time_per_epoch = [], []

        start_time = time.time()
        prev_subset_idx = None

        T = round(cfg.T0, 4)

        for epoch in range(1, self.epochs + 1):
            epoch_start_time = time.time()

            is_revision = cfg.revision_flag and (epoch > self.epochs - cfg.re
vision_epochs)

            if is_revision:
                T_epoch = 1.0000
            else:
                T_epoch = round(T, 4)

            subset_size = int(round(T_epoch * float(self.data_size)))
            subset_size = max(subset_size, self.batch_size)
            subset_size = min(subset_size, self.data_size)

            loader, subset_idx = self._build_subset_loader(subset_size, seed=
cfg.seed + epoch)

            if prev_subset_idx is not None:
                s_curr = set(subset_idx)
                s_prev = set(prev_subset_idx)
                inter = len(s_curr & s_prev)
                union = len(s_curr | s_prev)
                jaccard = inter / union if union > 0 else 0.0
                jaccard = round(jaccard, 4)
                print(
                    f"[DATA] Epoch {epoch}: Jaccard(prev_subset) = {jaccard:.
4f} "
                    f"(|curr|={len(s_curr)}, |prev|={len(s_prev)}, |n|={inte
r})"
                )
```

```
            else:
                print(f"[DATA] Epoch {epoch}: Jaccard(prev_subset) = N/A (fir
st epoch)")

            prev_subset_idx = subset_idx

            print(f"\nEpoch [{epoch}/{self.epochs}] T_epoch={T_epoch:.4f} sub
set={subset_size}")
            self.model.train()
            running_loss, correct, total = 0.0, 0, 0

            for inputs, labels in tqdm(loader, desc="Training", leave=False):
                inputs, labels = inputs.to(self.device), labels.to(self.devic
e)
                optimizer.zero_grad()
                outputs = self.model(inputs)
                loss = criterion(outputs, labels) / self.batch_size  # gradie
nt scale fix: normalise by original batch size
                loss.backward()
                optimizer.step()
                scheduler.step()   # step-based: counts optimizer steps, not
epochs
                running_loss += loss.item()
                correct += (outputs.argmax(1) == labels).sum().item()
                total += labels.size(0)
                total_bp += labels.size(0)

            train_acc = correct / max(total, 1)
            epoch_loss = running_loss / max(len(loader), 1)
            train_acc = round(train_acc, 4)
            epoch_loss = round(epoch_loss, 4)

            samples_used_per_epoch.append(subset_size)
            epoch_accuracies.append(train_acc)
            epoch_losses.append(epoch_loss)

            test_acc, val_loss = self._eval()
            # scheduler.step() removed here step-based decay runs inside the
batch loop above
            epoch_test_accuracies.append(test_acc)
            epoch_test_losses.append(val_loss)

            print(f"TrainAcc={train_acc:.4f} TestAcc={test_acc:.4f}")
```

```
            if not is_revision and epoch < self.epochs:
                delta_raw = train_acc - prev_acc
                delta = round(delta_raw, 4)
                accept = False
                reason = "none"

                if delta > cfg.delta_threshold:
                    accept = True
                    reason = "threshold"
                else:
                    r_value = rng.random()

                    boltz_prob_raw = math.exp(
                        cfg.boltz_scalar * delta / T
                    )
                    accept_prob = boltz_prob_raw

                    print(
                        f"[SA][DEBUG] epoch={epoch} delta={delta:+.4f}, "
                        f"T={T:.4f}, r={r_value:.4f}, "
                        f"boltz_prob_raw={boltz_prob_raw:.4f}, "
                        f"accept_prob={accept_prob:.4f}, "
                        f"scalar={cfg.boltz_scalar:.4f}"
                    )

                    if r_value < accept_prob:
                        accept = True
                        reason = "metropolis"
                    else:
                        accept = False
                        reason = "reheat"

                if accept:
                    T_old = T
                    T_new = T_base(epoch + 1)
                    T = round(T_new, 4)
                    print(
                        f"[SA][ACCEPT:{reason}] epoch={epoch} delta={delta:+.
4f} "
                        f"T: {T_old:.4f} → {T:.4f}"
                    )
                else:
                    T_old = T
                    T_new = min(cfg.T0, T * SA_REHEAT_MULT)
```

```
                T = round(T_new, 4)
                print(
                    f"[SA][REHEAT] epoch={epoch} delta={delta:+.4f} "
                    f"T: {T_old:.4f} → {T:.4f}"
                )

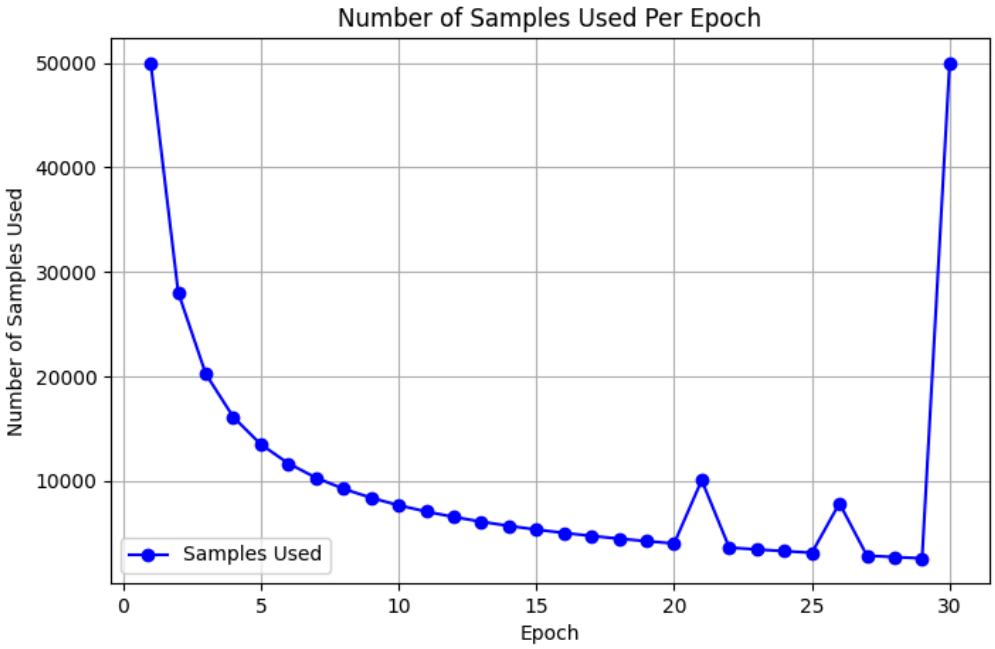
            prev_acc = train_acc
            epoch_time = time.time() - epoch_start_time
            epoch_time = round(epoch_time, 4)
            time_per_epoch.append(epoch_time)
            print(f"Epoch time: {epoch_time:.4f}s")

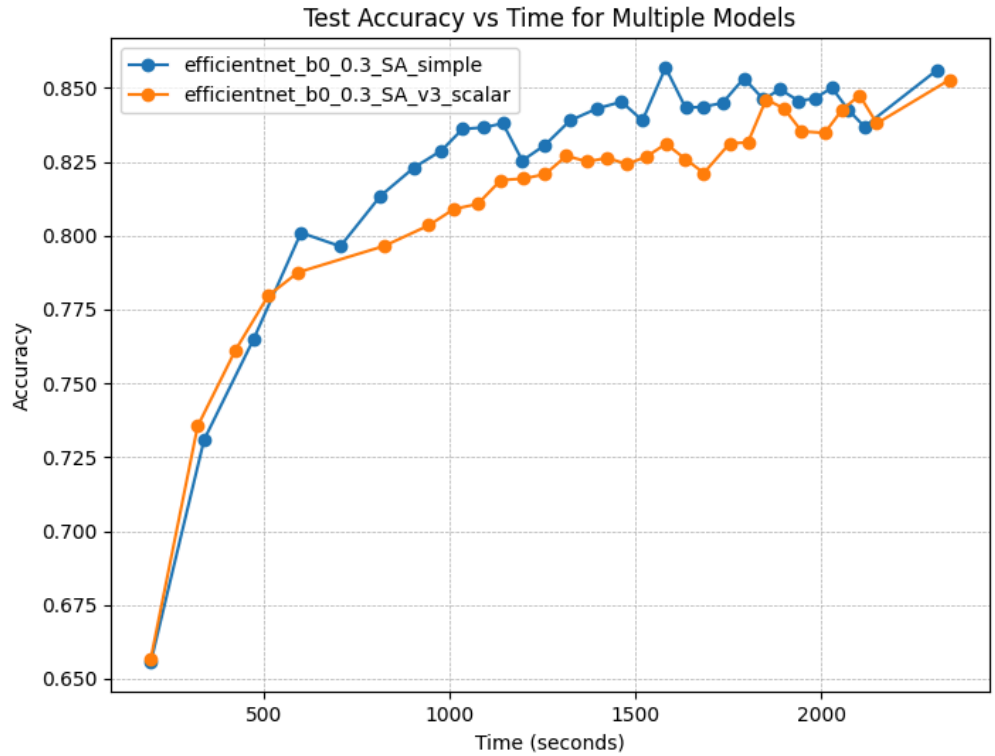
        end_time = time.time()
        log_memory(start_time, end_time)

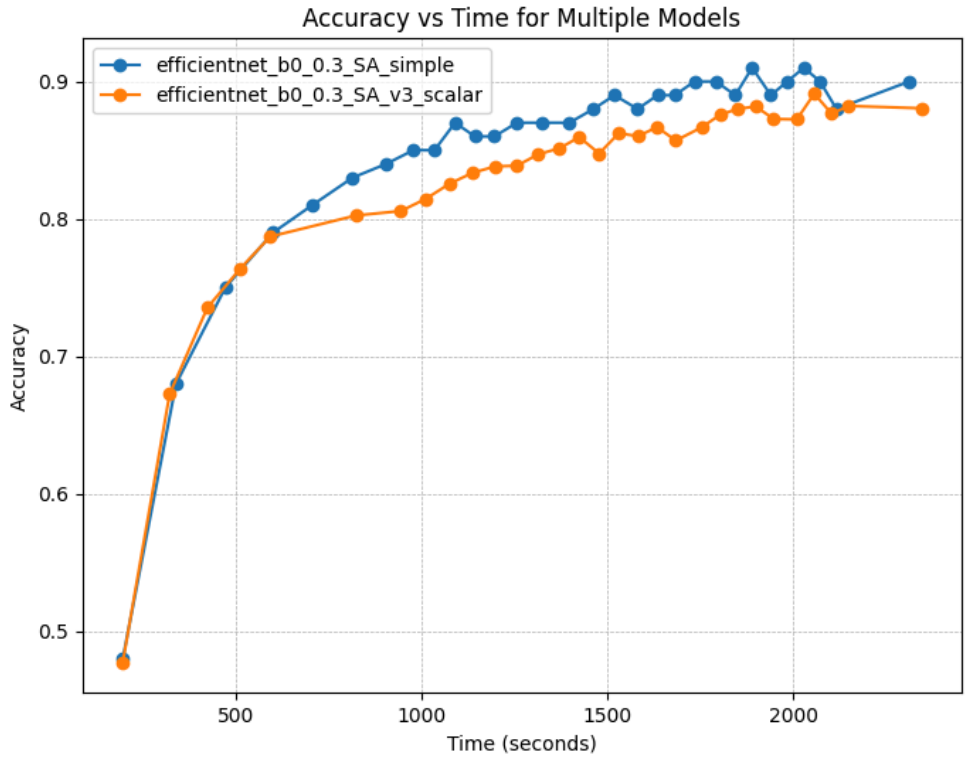
        eff_epochs = total_bp / float(self.data_size)
        eff_epochs = round(eff_epochs, 4)
        print(f"\nTotal backprop samples: {total_bp}")
        print(f"Effective epochs (float): {eff_epochs:.4f}")

        plot_accuracy_time_multi(
            model_name=self.model_name + "_SA_v3_scalar",
            accuracy=epoch_accuracies,
            time_per_epoch=time_per_epoch,
            save_path=self.save_path,
            data_file=self.save_path,
        )
        plot_accuracy_time_multi_test(
            model_name=self.model_name + "_SA_v3_scalar",
            accuracy=epoch_test_accuracies,
            time_per_epoch=time_per_epoch,
            samples_per_epoch=samples_used_per_epoch,
            threshold=0.0,
            save_path=self.save_path,
            data_file=self.save_path,
        )

        num_step = int(total_bp)
        return self.model, num_step
```


Number of Samples Used Per Epoch
Number of Samples Used
Epoch
Samples Used
50000
40000
30000
20000
10000
0
5
10
15
20
25
30


Test Accuracy vs Time for Multiple Models
efficientnet_b0_0.3_SA_simple
efficientnet_b0_0.3_SA_v3_scalar
Accuracy
Time (seconds)
0.850
0.825
0.800
0.775
0.750
0.725
0.700
0.675
0.650
500
1000
1500
2000



train_with_sa 30 epochs mobilenet2

(venv) ➜ energy_eff_training python ./main.py \
--model mobilenet_v2 \

--mode train_with_sa \

--epoch 30 \
--save_path cifar10_results_30epochs/mobilenet_v2 \

--dataset cifar10 \
--batch_size 32 \
--start_revision 0 \
--task classification \
--threshold 0.3 \
--seed 123
Using device: mps
Apple Silicon GPU (MPS) enabled.
/Users/amar/Developer/energy_eff_training/venv/lib/python3.13/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)

entry = pickle.load(f, encoding="latin1")
[DATA] Epoch 1: Jaccard(prev_subset) = N/A (first epoch)

Epoch [1/30] T_epoch=1.0000 subset=50000
Training: 0%|
| 0/1563 [00:00<?, ?it/s]/Users/amar/Developer/energy_eff_training/venv/lib/python3.13/site-packages/torch/utils/data/dataloader.py:1118: UserWarning: 'pin_memory' argument is set as true but not supported on MPS now, device pinned memory won't be used.
super().**init**(loader)
TrainAcc=0.4641 TestAcc=0.5967

[SA][ACCEPT:threshold] epoch=1 delta=+0.4641 T: 1.0000 → 0.5607
Epoch time: 122.8354s
[DATA] Epoch 2: Jaccard(prev_subset) = 0.5607 (|curr|=28035, |prev|=50000, |∩|=28035)

Epoch [2/30] T_epoch=0.5607 subset=28035
TrainAcc=0.6312 TestAcc=0.6516

[SA][ACCEPT:threshold] epoch=2 delta=+0.1671 T: 0.5607 → 0.4049
Epoch time: 83.8787s
[DATA] Epoch 3: Jaccard(prev_subset) = 0.3086 (|curr|=20245, |prev|=28035, |∩|=11387)

Epoch [3/30] T_epoch=0.4049 subset=20245
TrainAcc=0.6794 TestAcc=0.6991

[SA][ACCEPT:threshold] epoch=3 delta=+0.0482 T: 0.4049 → 0.3224
Epoch time: 70.4958s
[DATA] Epoch 4: Jaccard(prev_subset) = 0.2163 (|curr|=16120, |prev|=20245, |∩|=6466)

Epoch [4/30] T_epoch=0.3224 subset=16120
TrainAcc=0.7182 TestAcc=0.7111

[SA][ACCEPT:threshold] epoch=4 delta=+0.0388 T: 0.3224 → 0.2702
Epoch time: 63.3280s
[DATA] Epoch 5: Jaccard(prev_subset) = 0.1743 (|curr|=13510, |prev|=16120, |∩|=4399)

Epoch [5/30] T_epoch=0.2702 subset=13510
TrainAcc=0.7370 TestAcc=0.7097

[SA][ACCEPT:threshold] epoch=5 delta=+0.0188 T: 0.2702 → 0.2337
Epoch time: 59.0178s
[DATA] Epoch 6: Jaccard(prev_subset) = 0.1455 (|curr|=11685, |prev|=13510, |∩|=3201)

Epoch [6/30] T_epoch=0.2337 subset=11685
TrainAcc=0.7534 TestAcc=0.7391

[SA][ACCEPT:threshold] epoch=6 delta=+0.0164 T: 0.2337 → 0.2065
Epoch time: 55.8756s
[DATA] Epoch 7: Jaccard(prev_subset) = 0.1204 (|curr|=10325, |prev|=11685, |∩|=2365)

Epoch [7/30] T_epoch=0.2065 subset=10325
TrainAcc=0.7592 TestAcc=0.7491

[SA][ACCEPT:threshold] epoch=7 delta=+0.0058 T: 0.2065 → 0.1852
Epoch time: 53.2198s
[DATA] Epoch 8: Jaccard(prev_subset) = 0.1057 (|curr|=9260, |prev|=10325, |∩|=1873)

Epoch [8/30] T_epoch=0.1852 subset=9260
TrainAcc=0.7817 TestAcc=0.7556

[SA][ACCEPT:threshold] epoch=8 delta=+0.0225 T: 0.1852 → 0.1681
Epoch time: 51.6482s
[DATA] Epoch 9: Jaccard(prev_subset) = 0.0943 (|curr|=8405, |prev|=9260, |∩|=1522)

Epoch [9/30] T_epoch=0.1681 subset=8405
TrainAcc=0.7905 TestAcc=0.7620

[SA][ACCEPT:threshold] epoch=9 delta=+0.0088 T: 0.1681 → 0.1538
Epoch time: 49.8692s
[DATA] Epoch 10: Jaccard(prev_subset) = 0.0873 (|curr|=7690, |prev|=8405, |∩|=1292)

Epoch [10/30] T_epoch=0.1538 subset=7690
TrainAcc=0.8022 TestAcc=0.7559

[SA][ACCEPT:threshold] epoch=10 delta=+0.0117 T: 0.1538 → 0.1418
Epoch time: 615.1884s
[DATA] Epoch 11: Jaccard(prev_subset) = 0.0809 (|curr|=7090, |prev|=7690, |∩|=1106)

Epoch [11/30] T_epoch=0.1418 subset=7090
TrainAcc=0.8006 TestAcc=0.7683

[SA][DEBUG] epoch=11 delta=-0.0016, T=0.1418, r=0.6824, boltz_prob_raw=0.9722, accept_prob=0.9722, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=11 delta=-0.0016 T: 0.1418 → 0.1314
Epoch time: 209.4172s
[DATA] Epoch 12: Jaccard(prev_subset) = 0.0730 (|curr|=6570, |prev|=7090, |∩|=929)

Epoch [12/30] T_epoch=0.1314 subset=6570
TrainAcc=0.8067 TestAcc=0.7801

[SA][ACCEPT:threshold] epoch=12 delta=+0.0061 T: 0.1314 → 0.1224
Epoch time: 80.1902s
[DATA] Epoch 13: Jaccard(prev_subset) = 0.0664 (|curr|=6120, |prev|=6570, |∩|=790)

Epoch [13/30] T_epoch=0.1224 subset=6120
TrainAcc=0.8240 TestAcc=0.7706

[SA][ACCEPT:threshold] epoch=13 delta=+0.0173 T: 0.1224 → 0.1144
Epoch time: 67.4864s
[DATA] Epoch 14: Jaccard(prev_subset) = 0.0612 (|curr|=5720, |prev|=6120, |∩|=683)

Epoch [14/30] T_epoch=0.1144 subset=5720
TrainAcc=0.8227 TestAcc=0.7876

[SA][DEBUG] epoch=14 delta=-0.0013, T=0.1144, r=0.0538, boltz_prob_raw=0.9720, accept_prob=0.9720, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=14 delta=-0.0013 T: 0.1144 → 0.1073

Epoch time: 254.1323s
[DATA] Epoch 15: Jaccard(prev_subset) = 0.0600 (|curr|=5365, |prev|=5720, |∩|=627)

Epoch [15/30] T_epoch=0.1073 subset=5365
TrainAcc=0.8354 TestAcc=0.7794

[SA][ACCEPT:threshold] epoch=15 delta=+0.0127 T: 0.1073 → 0.1009
Epoch time: 155.0957s
[DATA] Epoch 16: Jaccard(prev_subset) = 0.0555 (|curr|=5045, |prev|=5365, |∩|=547)

Epoch [16/30] T_epoch=0.1009 subset=5045
TrainAcc=0.8264 TestAcc=0.7837

[SA][DEBUG] epoch=16 delta=-0.0090, T=0.1009, r=0.2204, boltz_prob_raw=0.8001, accept_prob=0.8001, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=16 delta=-0.0090 T: 0.1009 → 0.0952
Epoch time: 748.4262s
[DATA] Epoch 17: Jaccard(prev_subset) = 0.0510 (|curr|=4760, |prev|=5045, |∩|=476)

Epoch [17/30] T_epoch=0.0952 subset=4760
TrainAcc=0.8326 TestAcc=0.7657

[SA][ACCEPT:threshold] epoch=17 delta=+0.0062 T: 0.0952 → 0.0899
Epoch time: 43.5271s
[DATA] Epoch 18: Jaccard(prev_subset) = 0.0483 (|curr|=4495, |prev|=4760, |∩|=426)

Epoch [18/30] T_epoch=0.0899 subset=4495
TrainAcc=0.8376 TestAcc=0.7742

[SA][DEBUG] epoch=18 delta=+0.0050, T=0.0899, r=0.1844, boltz_prob_raw=1.1492, accept_prob=1.1492, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=18 delta=+0.0050 T: 0.0899 → 0.0852
Epoch time: 43.3479s
[DATA] Epoch 19: Jaccard(prev_subset) = 0.0433 (|curr|=4260, |prev|=4495, |∩|=363)

Epoch [19/30] T_epoch=0.0852 subset=4260
TrainAcc=0.8369 TestAcc=0.7905

[SA][DEBUG] epoch=19 delta=-0.0007, T=0.0852, r=0.1759, boltz_prob_raw=0.9797, accept_prob=0.9797, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=19 delta=-0.0007 T: 0.0852 → 0.0807
Epoch time: 42.9392s
[DATA] Epoch 20: Jaccard(prev_subset) = 0.0431 (|curr|=4035, |prev|=4260, |∩|=343)

Epoch [20/30] T_epoch=0.0807 subset=4035
TrainAcc=0.8337 TestAcc=0.7564

[SA][DEBUG] epoch=20 delta=-0.0032, T=0.0807, r=0.8121, boltz_prob_raw=0.9056, accept_prob=0.9056, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=20 delta=-0.0032 T: 0.0807 → 0.0767
Epoch time: 42.2746s
[DATA] Epoch 21: Jaccard(prev_subset) = 0.0410 (|curr|=3835, |prev|=4035, |∩|=310)

Epoch [21/30] T_epoch=0.0767 subset=3835
TrainAcc=0.8342 TestAcc=0.7932

[SA][DEBUG] epoch=21 delta=+0.0005, T=0.0767, r=0.9233, boltz_prob_raw=1.0164, accept_prob=1.0164, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=21 delta=+0.0005 T: 0.0767 → 0.0729
Epoch time: 42.2789s
[DATA] Epoch 22: Jaccard(prev_subset) = 0.0357 (|curr|=3645, |prev|=3835, |∩|=258)

Epoch [22/30] T_epoch=0.0729 subset=3645
TrainAcc=0.8491 TestAcc=0.7941

[SA][ACCEPT:threshold] epoch=22 delta=+0.0149 T: 0.0729 → 0.0693
Epoch time: 41.9072s
[DATA] Epoch 23: Jaccard(prev_subset) = 0.0339 (|curr|=3465, |prev|=3645, |∩|=233)

Epoch [23/30] T_epoch=0.0693 subset=3465
TrainAcc=0.8566 TestAcc=0.7947

[SA][ACCEPT:threshold] epoch=23 delta=+0.0075 T: 0.0693 → 0.0661
Epoch time: 41.5607s
[DATA] Epoch 24: Jaccard(prev_subset) = 0.0363 (|curr|=3305, |prev|=3465, |∩|=237)

Epoch [24/30] T_epoch=0.0661 subset=3305
TrainAcc=0.8433 TestAcc=0.7949

[SA][DEBUG] epoch=24 delta=-0.0133, T=0.0661, r=0.2766, boltz_prob_raw=0.6047, accept_prob=0.6047, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=24 delta=-0.0133 T: 0.0661 → 0.0630
Epoch time: 41.1495s
[DATA] Epoch 25: Jaccard(prev_subset) = 0.0336 (|curr|=3150, |prev|=3305, |∩|=210)

Epoch [25/30] T_epoch=0.0630 subset=3150
TrainAcc=0.8406 TestAcc=0.7872

[SA][DEBUG] epoch=25 delta=-0.0027, T=0.0630, r=0.8198, boltz_prob_raw=0.8984, accept_prob=0.8984, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=25 delta=-0.0027 T: 0.0630 → 0.0601
Epoch time: 41.1559s
[DATA] Epoch 26: Jaccard(prev_subset) = 0.0320 (|curr|=3005, |prev|=3150, |∩|=191)

Epoch [26/30] T_epoch=0.0601 subset=3005
TrainAcc=0.8602 TestAcc=0.7910

[SA][ACCEPT:threshold] epoch=26 delta=+0.0196 T: 0.0601 → 0.0573
Epoch time: 40.6102s
[DATA] Epoch 27: Jaccard(prev_subset) = 0.0322 (|curr|=2865, |prev|=3005, |∩|=183)

Epoch [27/30] T_epoch=0.0573 subset=2865
TrainAcc=0.8583 TestAcc=0.7983

[SA][DEBUG] epoch=27 delta=-0.0019, T=0.0573, r=0.8899, boltz_prob_raw=0.9204, accept_prob=0.9204, scalar=2.5000

[SA][ACCEPT:metropolis] epoch=27 delta=-0.0019 T: 0.0573 → 0.0548
Epoch time: 40.6041s
[DATA] Epoch 28: Jaccard(prev_subset) = 0.0277 (|curr|=2740, |prev|=2865, |∩|=151)

Epoch [28/30] T_epoch=0.0548 subset=2740
TrainAcc=0.8588 TestAcc=0.7927

[SA][DEBUG] epoch=28 delta=+0.0005, T=0.0548, r=0.5130, boltz_prob_raw=1.0231,
accept_prob=1.0231, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=28 delta=+0.0005 T: 0.0548 → 0.0523
Epoch time: 40.3687s
[DATA] Epoch 29: Jaccard(prev_subset) = 0.0239 (|curr|=2615, |prev|=2740, |∩|=125)

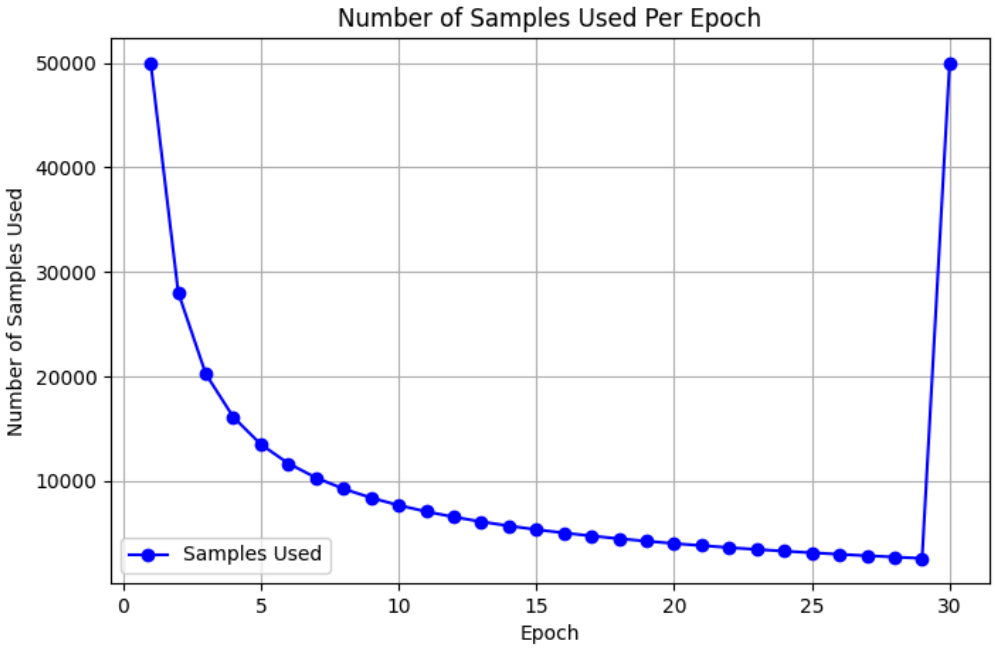
Epoch [29/30] T_epoch=0.0523 subset=2615
TrainAcc=0.8730 TestAcc=0.7938

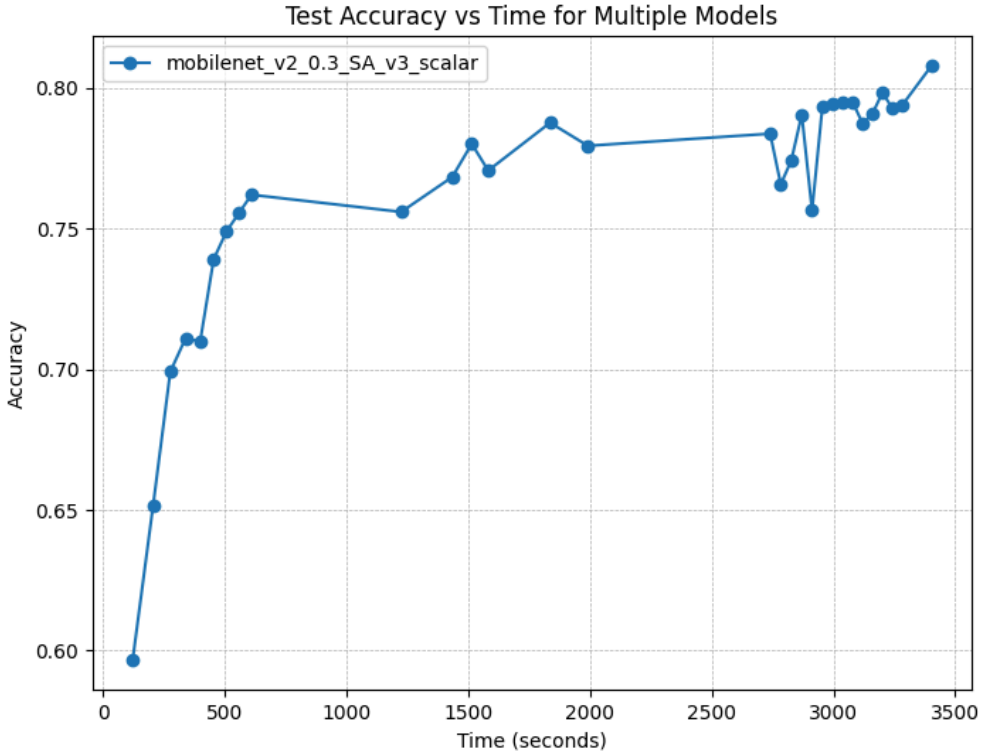
[SA][ACCEPT:threshold] epoch=29 delta=+0.0142 T: 0.0523 → 0.0500
Epoch time: 40.2008s
[DATA] Epoch 30: Jaccard(prev_subset) = 0.0523 (|curr|=50000, |prev|=2615, |∩|=2615)

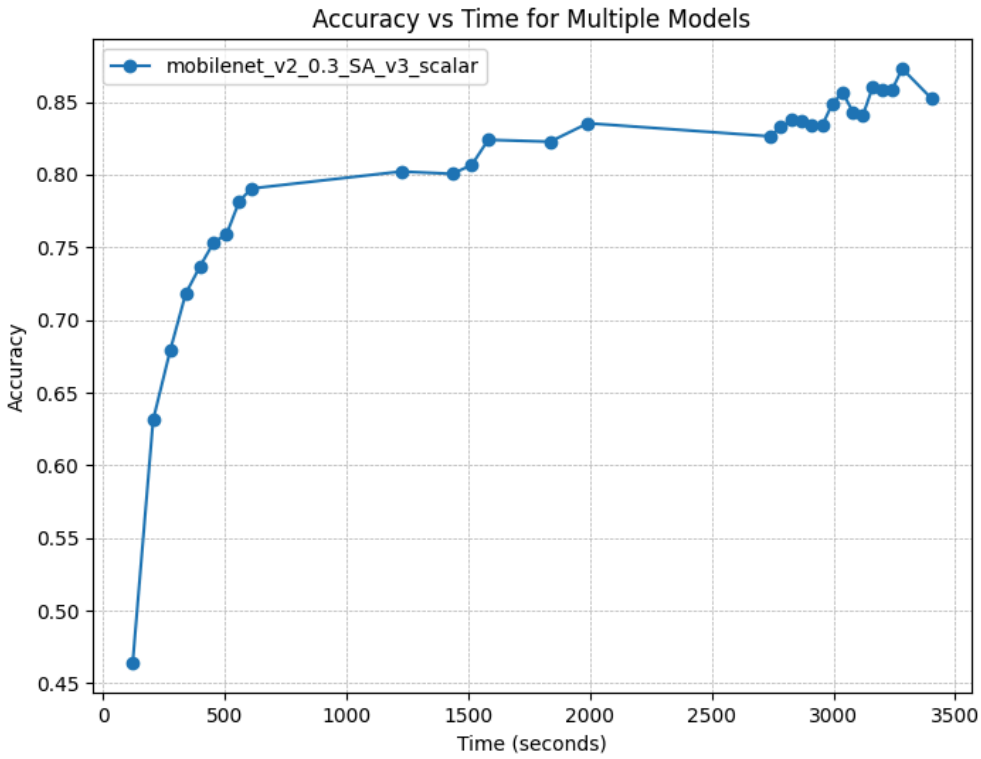
Epoch [30/30] T_epoch=1.0000 subset=50000
TrainAcc=0.8523 TestAcc=0.8079

Epoch time: 121.0749s
Training Time: 3403.10 seconds
Memory Consumption: 3815.09 MB

Total backprop samples: 307360
Effective epochs (float): 6.1472
Info: JSON file 'cifar10_results_30epochs/mobilenet_v2' not found. Initializing fresh data.
Info: Data for 'mobilenet_v2_0.3_SA_v3_scalar' saved to 'cifar10_results_30epochs/mobilenet_v2'.
Info: Plot saved to 'cifar10_results_30epochs/mobilenet_v2'.
Info: Test data JSON file 'cifar10_results_30epochs/mobilenet_v2_test' not found. Initializing fresh data.
Info: Test data for 'mobilenet_v2_0.3_SA_v3_scalar' saved to
'cifar10_results_30epochs/mobilenet_v2_test'.
Info: Test accuracy plot saved to 'cifar10_results_30epochs/mobilenet_v2_test'.
Info: Epoch samples plot saved to 'cifar10_results_30epochs/mobilenet_v2_test_epochs_00.png'.
Number of steps: 307360
Effective Epochs: 6


Number of Samples Used Per Epoch
Number of Samples Used
Epoch
Samples Used
50000
40000
30000
20000
10000
0
5
10
15
20
25
30


Test Accuracy vs Time for Multiple Models
mobilenet_v2_0.3_SA_v3_scalar
Accuracy
Time (seconds)
0.80
0.75
0.70
0.65
0.60
0
500
1000
1500
2000
2500
3000
3500



sa revision simple 30 epochs mobilenet v2

(venv) ➜ energy_eff_training python ./main.py \
--model mobilenet_v2 \
--mode saRevisionSimple \
--epoch 30 \
--save_path cifar10_results_30epochs/mobilenet_v2 \
--dataset cifar10 \
--batch_size 32 \
--start_revision 0 \
--task classification \
--threshold 0.3 \
--seed 123
Using device: mps
Apple Silicon GPU (MPS) enabled.
/Users/amar/Developer/energy_eff_training/venv/lib/python3.13/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
entry = pickle.load(f, encoding="latin1")

Epoch [1/30] α=2.00 T=1.0000 subset=50000
/Users/amar/Developer/energy_eff_training/saRevisionSimple.py:234: UserWarning: The epoch parameter in `scheduler.step()` was not necessary and is being deprecated where possible. Please use `scheduler.step()` to step the scheduler. During the deprecation, if epoch is different from None, the closed form is used instead of the new chainable form, where available. Please open an issue if you are unable to replicate your use case: https://github.com/pytorch/pytorch/issues/new/choose.
scheduler.step(val_loss)
TrainAcc=0.47 TrainLoss=1.4419 | TestAcc=0.5930 TestLoss=1.1307
[SA] δ=+0.47 r=- boltz=- T=1.0000 → ACCEPT (improvement)
[SA] proposed α=1.00 accepted
Epoch time: 121.41s

Epoch [2/30] α=1.00 T=0.8532 subset=33333
TrainAcc=0.64 TrainLoss=1.0113 | TestAcc=0.6704 TestLoss=0.9294

[SA] δ=+0.17 r=- boltz=- T=0.8532 → ACCEPT (improvement)
[SA] proposed α=2.00 accepted
Epoch time: 92.27s

Epoch [3/30] α=2.00 T=0.7279 subset=30000
TrainAcc=0.70 TrainLoss=0.8578 | TestAcc=0.7099 TestLoss=0.8252

[SA] δ=+0.06 r=- boltz=- T=0.7279 → ACCEPT (improvement)
[SA] proposed α=3.00 accepted
Epoch time: 86.51s

Epoch [4/30] α=3.00 T=0.6210 subset=28571
TrainAcc=0.75 TrainLoss=0.7202 | TestAcc=0.7350 TestLoss=0.7576

[SA] δ=+0.05 r=- boltz=- T=0.6210 → ACCEPT (improvement)
[SA] proposed α=2.00 accepted
Epoch time: 84.65s

Epoch [5/30] α=2.00 T=0.5298 subset=21428
TrainAcc=0.78 TrainLoss=0.6390 | TestAcc=0.7627 TestLoss=0.6714

[SA] δ=+0.03 r=- boltz=- T=0.5298 → ACCEPT (improvement)
[SA] proposed α=3.00 accepted
Epoch time: 72.56s

Epoch [6/30] α=3.00 T=0.4520 subset=22222
TrainAcc=0.80 TrainLoss=0.5832 | TestAcc=0.7758 TestLoss=0.6630

[SA] δ=+0.02 r=- boltz=- T=0.4520 → ACCEPT (improvement)
[SA] proposed α=2.00 accepted
Epoch time: 73.72s

Epoch [7/30] α=2.00 T=0.3857 subset=16666
TrainAcc=0.82 TrainLoss=0.5377 | TestAcc=0.7745 TestLoss=0.6604

[SA] δ=+0.02 r=- boltz=- T=0.3857 → ACCEPT (improvement)
[SA] proposed α=1.00 accepted

Epoch time: 64.23s

Epoch [8/30] α=1.00 T=0.3290 subset=11111
TrainAcc=0.83 TrainLoss=0.4976 | TestAcc=0.7809 TestLoss=0.6619

[SA] δ=+0.01 r=- boltz=- T=0.3290 → ACCEPT (improvement)
[SA] proposed α=0.10 accepted
Epoch time: 54.85s

Epoch [9/30] α=0.10 T=0.2807 subset=6043
TrainAcc=0.83 TrainLoss=0.4784 | TestAcc=0.7967 TestLoss=0.5938

[SA] δ=+0.00 r=0.81 boltz=1.0 T=0.2807 → ACCEPT (Metropolis)
[SA] proposed α=0.10 accepted
Epoch time: 45.84s

Epoch [10/30] α=0.10 T=0.2395 subset=5445
TrainAcc=0.84 TrainLoss=0.4753 | TestAcc=0.7855 TestLoss=0.6450

[SA] δ=+0.01 r=- boltz=- T=0.2395 → ACCEPT (improvement)
[SA] proposed α=0.10 accepted
Epoch time: 44.85s

Epoch [11/30] α=0.10 T=0.2043 subset=4954
TrainAcc=0.83 TrainLoss=0.4985 | TestAcc=0.7689 TestLoss=0.6873

[SA] δ=-0.01 r=0.28 boltz=0.95 T=0.2043 → ACCEPT (Metropolis)
[SA] proposed α=0.10 accepted
Epoch time: 43.88s

Epoch [12/30] α=0.10 T=0.1743 subset=4545
TrainAcc=0.85 TrainLoss=0.4545 | TestAcc=0.7969 TestLoss=0.5982

[SA] δ=+0.02 r=- boltz=- T=0.1743 → ACCEPT (improvement)
[SA] proposed α=1.10 accepted
Epoch time: 43.69s

Epoch [13/30] α=1.10 T=0.1487 subset=7446
TrainAcc=0.84 TrainLoss=0.4584 | TestAcc=0.7946 TestLoss=0.6164

[SA] δ=-0.01 r=0.89 boltz=0.93 T=0.1487 → ACCEPT (Metropolis)
[SA] proposed α=2.10 accepted
Epoch time: 48.38s

Epoch [14/30] α=2.10 T=0.1269 subset=9627
TrainAcc=0.86 TrainLoss=0.4276 | TestAcc=0.8110 TestLoss=0.5575

[SA] δ=+0.02 r=- boltz=- T=0.1269 → ACCEPT (improvement)
[SA] proposed α=3.10 accepted
Epoch time: 51.82s

Epoch [15/30] α=3.10 T=0.1083 subset=11325
TrainAcc=0.86 TrainLoss=0.4132 | TestAcc=0.8028 TestLoss=0.5835

[SA] δ=+0.00 r=0.24 boltz=1.0 T=0.1083 → ACCEPT (Metropolis)
[SA] proposed α=2.10 accepted
Epoch time: 54.99s

Epoch [16/30] α=2.10 T=0.0924 subset=8563
TrainAcc=0.87 TrainLoss=0.3969 | TestAcc=0.8045 TestLoss=0.6040

[SA] δ=+0.01 r=- boltz=- T=0.0924 → ACCEPT (improvement)
[SA] proposed α=3.10 accepted
Epoch time: 50.05s

Epoch [17/30] α=3.10 T=0.0788 subset=10199
TrainAcc=0.88 TrainLoss=0.3692 | TestAcc=0.7990 TestLoss=0.6075

[SA] δ=+0.01 r=- boltz=- T=0.0788 → ACCEPT (improvement)
[SA] proposed α=2.10 accepted
Epoch time: 52.91s

Epoch [18/30] α=2.10 T=0.0672 subset=7711
TrainAcc=0.87 TrainLoss=0.3812 | TestAcc=0.8108 TestLoss=0.5711

[SA] δ=-0.01 r=0.21 boltz=0.86 T=0.0672 → ACCEPT (Metropolis)
[SA] proposed α=3.10 accepted
Epoch time: 48.84s

Epoch [19/30] α=3.10 T=0.0574 subset=9276
TrainAcc=0.87 TrainLoss=0.3814 | TestAcc=0.8126 TestLoss=0.5658

[SA] δ=+0.00 r=0.63 boltz=1.0 T=0.0574 → ACCEPT (Metropolis)
[SA] proposed α=2.10 accepted
Epoch time: 51.57s

Epoch [20/30] α=2.10 T=0.0489 subset=7013
TrainAcc=0.87 TrainLoss=0.3586 | TestAcc=0.8122 TestLoss=0.5650

[SA] δ=+0.00 r=0.93 boltz=1.0 T=0.0489 → ACCEPT (Metropolis)
[SA] proposed α=3.10 accepted
Epoch time: 47.39s

Epoch [21/30] α=3.10 T=0.0418 subset=8506
TrainAcc=0.88 TrainLoss=0.3557 | TestAcc=0.8193 TestLoss=0.5659

[SA] δ=+0.01 r=- boltz=- T=0.0418 → ACCEPT (improvement)
[SA] proposed α=4.10 accepted
Epoch time: 49.99s

Epoch [22/30] α=4.10 T=0.0356 subset=9770
TrainAcc=0.89 TrainLoss=0.3271 | TestAcc=0.8106 TestLoss=0.5761

[SA] δ=+0.01 r=- boltz=- T=0.0356 → ACCEPT (improvement)
[SA] proposed α=3.10 accepted
Epoch time: 158.97s

Epoch [23/30] α=3.10 T=0.0304 subset=7854
TrainAcc=0.90 TrainLoss=0.3062 | TestAcc=0.8177 TestLoss=0.5580

[SA] δ=+0.01 r=- boltz=- T=0.0304 → ACCEPT (improvement)
[SA] proposed α=4.10 accepted
Epoch time: 83.26s

Epoch [24/30] α=4.10 T=0.0259 subset=9074
TrainAcc=0.89 TrainLoss=0.3242 | TestAcc=0.8197 TestLoss=0.5605

[SA] δ=-0.01 r=0.52 boltz=0.68 T=0.0259 → ACCEPT (Metropolis)
[SA] proposed α=5.10 accepted
Epoch time: 256.04s

Epoch [25/30] α=5.10 T=0.0221 subset=10132
TrainAcc=0.89 TrainLoss=0.3102 | TestAcc=0.8203 TestLoss=0.5584

[SA] δ=+0.00 r=0.17 boltz=1.0 T=0.0221 → ACCEPT (Metropolis)
[SA] proposed α=6.10 accepted
Epoch time: 70.64s

Epoch [26/30] α=6.10 T=0.0189 subset=11059
TrainAcc=0.89 TrainLoss=0.3103 | TestAcc=0.8177 TestLoss=0.5571

[SA] δ=+0.00 r=0.5 boltz=1.0 T=0.0189 → ACCEPT (Metropolis)
[SA] proposed α=5.10 accepted
Epoch time: 138.23s

Epoch [27/30] α=5.10 T=0.0161 subset=9501
TrainAcc=0.90 TrainLoss=0.2879 | TestAcc=0.8276 TestLoss=0.5269

[SA] δ=+0.01 r=- boltz=- T=0.0161 → ACCEPT (improvement)
[SA] proposed α=4.10 accepted
Epoch time: 52.36s

Epoch [28/30] α=4.10 T=0.0137 subset=7943
TrainAcc=0.91 TrainLoss=0.2728 | TestAcc=0.8095 TestLoss=0.6131

[SA] δ=+0.01 r=- boltz=- T=0.0137 → ACCEPT (improvement)
[SA] proposed α=5.10 accepted
Epoch time: 49.41s

Epoch [29/30] α=5.10 T=0.0117 subset=8944
TrainAcc=0.91 TrainLoss=0.2556 | TestAcc=0.8308 TestLoss=0.5260

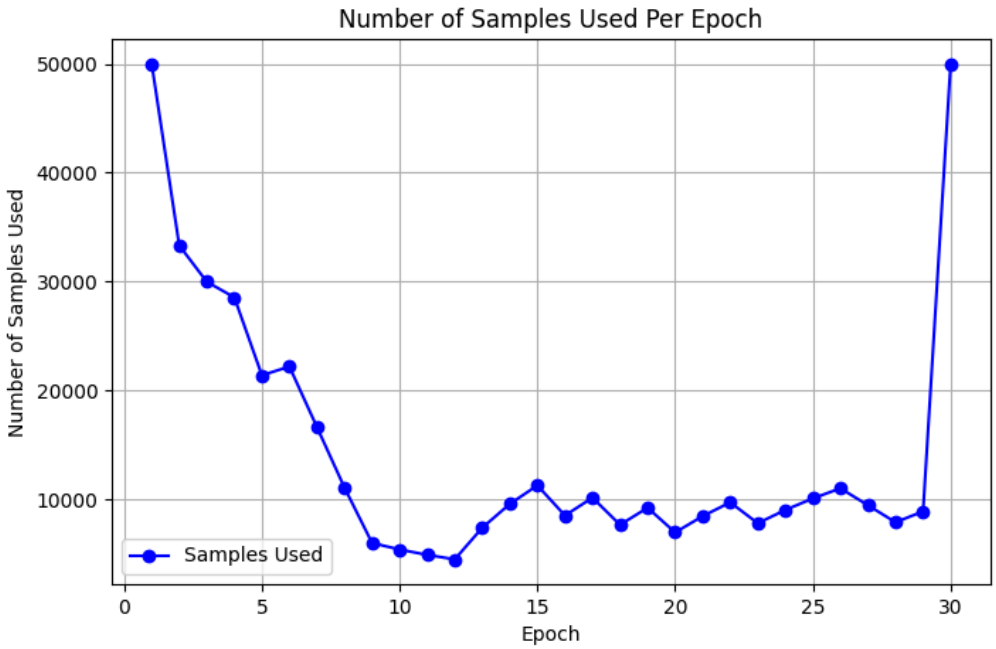
[SA] δ=+0.00 r=0.01 boltz=1.0 T=0.0117 → ACCEPT (Metropolis)
[SA] proposed α=4.10 accepted
Epoch time: 51.08s

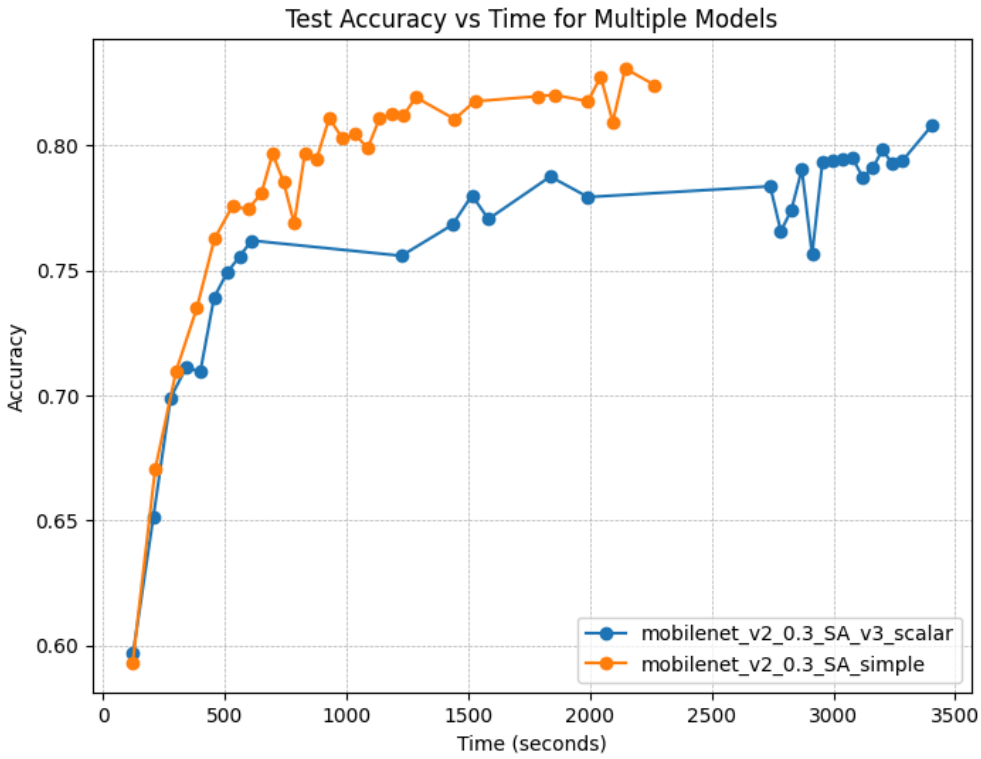
Epoch [30/30] α=4.10 T=0.0100 subset=50000
TrainAcc=0.90 TrainLoss=0.2934 | TestAcc=0.8242 TestLoss=0.5468

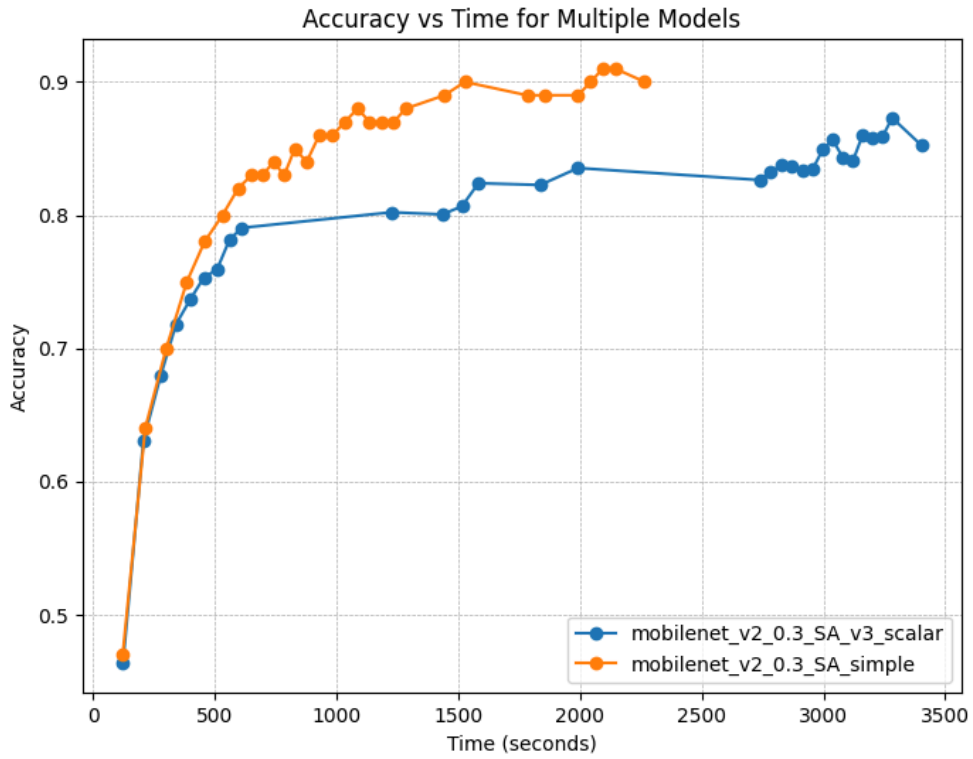
Epoch time: 121.50s
Training Time: 2265.88 seconds

Memory Consumption: 3588.30 MB

Total backprop samples: 438261
Effective epochs (float): 8.765
Info: Data for 'mobilenet_v2_0.3_SA_simple' saved to 'cifar10_results_30epochs/mobilenet_v2'.
Info: Plot saved to 'cifar10_results_30epochs/mobilenet_v2'.
Info: Test data for 'mobilenet_v2_0.3_SA_simple' saved to 'cifar10_results_30epochs/mobilenet_v2_test'.
Info: Test accuracy plot saved to 'cifar10_results_30epochs/mobilenet_v2_test'.
Info: Epoch samples plot saved to 'cifar10_results_30epochs/mobilenet_v2_test_epochs_00.png'.
Number of steps: 438261
Effective Epochs: 8




Test Accuracy vs Time for Multiple Models
Accuracy
Time (seconds)
0.60
0.65
0.70
0.75
0.80
0
500
1000
1500
2000
2500
3000
3500
mobilenet_v2_0.3_SA_v3_scalar
mobilenet_v2_0.3_SA_simple



sa revision simple 30 epochs resnet 50

(venv) ➜ energy_eff_training python ./main.py \
--model resnet50 \

--mode saRevisionSimple \
--epoch 30 \
--save_path cifar10_results_30epochs/resnet50 \

--dataset cifar10 \
--batch_size 32 \
--start_revision 0 \
--task classification \
--threshold 0.3 \
--seed 123
Using device: mps
Apple Silicon GPU (MPS) enabled.
/Users/amar/Developer/energy_eff_training/venv/lib/python3.13/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
entry = pickle.load(f, encoding="latin1")

Epoch [1/30] α=2.00 T=1.0000 subset=50000
/Users/amar/Developer/energy_eff_training/saRevisionSimple.py:234: UserWarning: The epoch parameter in `scheduler.step()` was not necessary and is being deprecated where possible. Please use `scheduler.step()` to step the scheduler. During the deprecation, if epoch is different from None, the closed form is used instead of the new chainable form, where available. Please open an issue if you are unable to replicate your use case: https://github.com/pytorch/pytorch/issues/new/choose.
scheduler.step(val_loss)
TrainAcc=0.47 TrainLoss=1.4625 | TestAcc=0.6149 TestLoss=1.0883
[SA] δ=+0.47 r=- boltz=- T=1.0000 → ACCEPT (improvement)
[SA] proposed α=1.00 accepted
Epoch time: 274.52s

Epoch [2/30] α=1.00 T=0.8532 subset=33333
TrainAcc=0.66 TrainLoss=0.9593 | TestAcc=0.6820 TestLoss=0.9387

[SA] δ=+0.19 r=- boltz=- T=0.8532 → ACCEPT (improvement)
[SA] proposed α=2.00 accepted
Epoch time: 196.72s

Epoch [3/30] α=2.00 T=0.7279 subset=30000
TrainAcc=0.73 TrainLoss=0.7878 | TestAcc=0.7349 TestLoss=0.7673

[SA] δ=+0.07 r=- boltz=- T=0.7279 → ACCEPT (improvement)
[SA] proposed α=3.00 accepted
Epoch time: 181.20s

Epoch [4/30] α=3.00 T=0.6210 subset=28571
TrainAcc=0.77 TrainLoss=0.6659 | TestAcc=0.7223 TestLoss=0.8288

[SA] δ=+0.04 r=- boltz=- T=0.6210 → ACCEPT (improvement)
[SA] proposed α=2.00 accepted
Epoch time: 174.55s

Epoch [5/30] α=2.00 T=0.5298 subset=21428
TrainAcc=0.80 TrainLoss=0.5831 | TestAcc=0.7514 TestLoss=0.7333

[SA] δ=+0.03 r=- boltz=- T=0.5298 → ACCEPT (improvement)
[SA] proposed α=3.00 accepted
Epoch time: 141.57s

Epoch [6/30] α=3.00 T=0.4520 subset=22222
TrainAcc=0.81 TrainLoss=0.5416 | TestAcc=0.7788 TestLoss=0.6590

[SA] δ=+0.01 r=- boltz=- T=0.4520 → ACCEPT (improvement)
[SA] proposed α=2.00 accepted
Epoch time: 145.02s

Epoch [7/30] α=2.00 T=0.3857 subset=16666
TrainAcc=0.83 TrainLoss=0.4965 | TestAcc=0.7906 TestLoss=0.6163

[SA] δ=+0.02 r=- boltz=- T=0.3857 → ACCEPT (improvement)
[SA] proposed α=1.00 accepted

Epoch time: 119.81s

Epoch [8/30] α=1.00 T=0.3290 subset=11111
TrainAcc=0.84 TrainLoss=0.4637 | TestAcc=0.8091 TestLoss=0.5707

[SA] δ=+0.01 r=- boltz=- T=0.3290 → ACCEPT (improvement)
[SA] proposed α=0.10 accepted
Epoch time: 94.75s

Epoch [9/30] α=0.10 T=0.2807 subset=6043
TrainAcc=0.85 TrainLoss=0.4437 | TestAcc=0.7881 TestLoss=0.6344

[SA] δ=+0.01 r=- boltz=- T=0.2807 → ACCEPT (improvement)
[SA] proposed α=0.10 accepted
Epoch time: 71.53s

Epoch [10/30] α=0.10 T=0.2395 subset=5445
TrainAcc=0.86 TrainLoss=0.4065 | TestAcc=0.7903 TestLoss=0.6267

[SA] δ=+0.01 r=- boltz=- T=0.2395 → ACCEPT (improvement)
[SA] proposed α=0.10 accepted
Epoch time: 69.38s

Epoch [11/30] α=0.10 T=0.2043 subset=4954
TrainAcc=0.85 TrainLoss=0.4480 | TestAcc=0.7858 TestLoss=0.6776

[SA] δ=-0.01 r=0.92 boltz=0.95 T=0.2043 → ACCEPT (Metropolis)
[SA] proposed α=0.10 accepted
Epoch time: 66.49s

Epoch [12/30] α=0.10 T=0.1743 subset=4545
TrainAcc=0.85 TrainLoss=0.4454 | TestAcc=0.8000 TestLoss=0.6006

[SA] δ=+0.00 r=0.28 boltz=1.0 T=0.1743 → ACCEPT (Metropolis)
[SA] proposed α=1.10 accepted
Epoch time: 65.98s

Epoch [13/30] α=1.10 T=0.1487 subset=7446
TrainAcc=0.85 TrainLoss=0.4282 | TestAcc=0.7725 TestLoss=0.7373

[SA] δ=+0.00 r=0.89 boltz=1.0 T=0.1487 → ACCEPT (Metropolis)
[SA] proposed α=2.10 accepted
Epoch time: 77.98s

Epoch [14/30] α=2.10 T=0.1269 subset=9627
TrainAcc=0.87 TrainLoss=0.3824 | TestAcc=0.8047 TestLoss=0.6204

[SA] δ=+0.02 r=- boltz=- T=0.1269 → ACCEPT (improvement)
[SA] proposed α=3.10 accepted
Epoch time: 87.62s

Epoch [15/30] α=3.10 T=0.1083 subset=11325
TrainAcc=0.87 TrainLoss=0.3829 | TestAcc=0.8116 TestLoss=0.5664

[SA] δ=+0.00 r=0.24 boltz=1.0 T=0.1083 → ACCEPT (Metropolis)
[SA] proposed α=2.10 accepted
Epoch time: 95.54s

Epoch [16/30] α=2.10 T=0.0924 subset=8563
TrainAcc=0.88 TrainLoss=0.3579 | TestAcc=0.8106 TestLoss=0.5996

[SA] δ=+0.01 r=- boltz=- T=0.0924 → ACCEPT (improvement)
[SA] proposed α=3.10 accepted
Epoch time: 83.12s

Epoch [17/30] α=3.10 T=0.0788 subset=10199
TrainAcc=0.89 TrainLoss=0.3278 | TestAcc=0.8219 TestLoss=0.5261

[SA] δ=+0.01 r=- boltz=- T=0.0788 → ACCEPT (improvement)
[SA] proposed α=2.10 accepted
Epoch time: 90.55s

Epoch [18/30] α=2.10 T=0.0672 subset=7711
TrainAcc=0.89 TrainLoss=0.3296 | TestAcc=0.8078 TestLoss=0.6239

[SA] δ=+0.00 r=0.21 boltz=1.0 T=0.0672 → ACCEPT (Metropolis)
[SA] proposed α=3.10 accepted
Epoch time: 392.09s

Epoch [19/30] α=3.10 T=0.0574 subset=9276
TrainAcc=0.89 TrainLoss=0.3301 | TestAcc=0.7939 TestLoss=0.6771

[SA] δ=+0.00 r=0.63 boltz=1.0 T=0.0574 → ACCEPT (Metropolis)
[SA] proposed α=2.10 accepted
Epoch time: 87.00s

Epoch [20/30] α=2.10 T=0.0489 subset=7013
TrainAcc=0.90 TrainLoss=0.3012 | TestAcc=0.8199 TestLoss=0.5702

[SA] δ=+0.01 r=- boltz=- T=0.0489 → ACCEPT (improvement)
[SA] proposed α=3.10 accepted
Epoch time: 75.94s

Epoch [21/30] α=3.10 T=0.0418 subset=8506
TrainAcc=0.89 TrainLoss=0.3225 | TestAcc=0.8141 TestLoss=0.5897

[SA] δ=-0.01 r=0.23 boltz=0.79 T=0.0418 → ACCEPT (Metropolis)
[SA] proposed α=2.10 accepted
Epoch time: 82.77s

Epoch [22/30] α=2.10 T=0.0356 subset=6431
TrainAcc=0.91 TrainLoss=0.2709 | TestAcc=0.8218 TestLoss=0.5868

[SA] δ=+0.02 r=- boltz=- T=0.0356 → ACCEPT (improvement)
[SA] proposed α=3.10 accepted
Epoch time: 73.41s

Epoch [23/30] α=3.10 T=0.0304 subset=7854
TrainAcc=0.91 TrainLoss=0.2711 | TestAcc=0.8208 TestLoss=0.5803

[SA] δ=+0.00 r=0.52 boltz=1.0 T=0.0304 → ACCEPT (Metropolis)
[SA] proposed α=4.10 accepted
Epoch time: 79.82s

Epoch [24/30] α=4.10 T=0.0259 subset=9074
TrainAcc=0.90 TrainLoss=0.2855 | TestAcc=0.8194 TestLoss=0.5979

[SA] δ=-0.01 r=0.23 boltz=0.68 T=0.0259 → ACCEPT (Metropolis)
[SA] proposed α=5.10 accepted
Epoch time: 85.62s

Epoch [25/30] α=5.10 T=0.0221 subset=10132
TrainAcc=0.90 TrainLoss=0.2717 | TestAcc=0.8179 TestLoss=0.5774

[SA] δ=+0.00 r=0.5 boltz=1.0 T=0.0221 → ACCEPT (Metropolis)
[SA] proposed α=4.10 accepted
Epoch time: 90.17s

Epoch [26/30] α=4.10 T=0.0189 subset=8471
TrainAcc=0.91 TrainLoss=0.2596 | TestAcc=0.8230 TestLoss=0.5594

[SA] δ=+0.01 r=- boltz=- T=0.0189 → ACCEPT (improvement)
[SA] proposed α=3.10 accepted
Epoch time: 82.63s

Epoch [27/30] α=3.10 T=0.0161 subset=6810
TrainAcc=0.92 TrainLoss=0.2320 | TestAcc=0.8362 TestLoss=0.5305

[SA] δ=+0.01 r=- boltz=- T=0.0161 → ACCEPT (improvement)
[SA] proposed α=2.10 accepted
Epoch time: 75.10s

Epoch [28/30] α=2.10 T=0.0137 subset=5149
TrainAcc=0.91 TrainLoss=0.2609 | TestAcc=0.8341 TestLoss=0.5422

[SA] δ=-0.01 r=0.18 boltz=0.48 T=0.0137 → ACCEPT (Metropolis)
[SA] proposed α=3.10 accepted
Epoch time: 67.60s

Epoch [29/30] α=3.10 T=0.0117 subset=6386
TrainAcc=0.92 TrainLoss=0.2361 | TestAcc=0.8229 TestLoss=0.5716

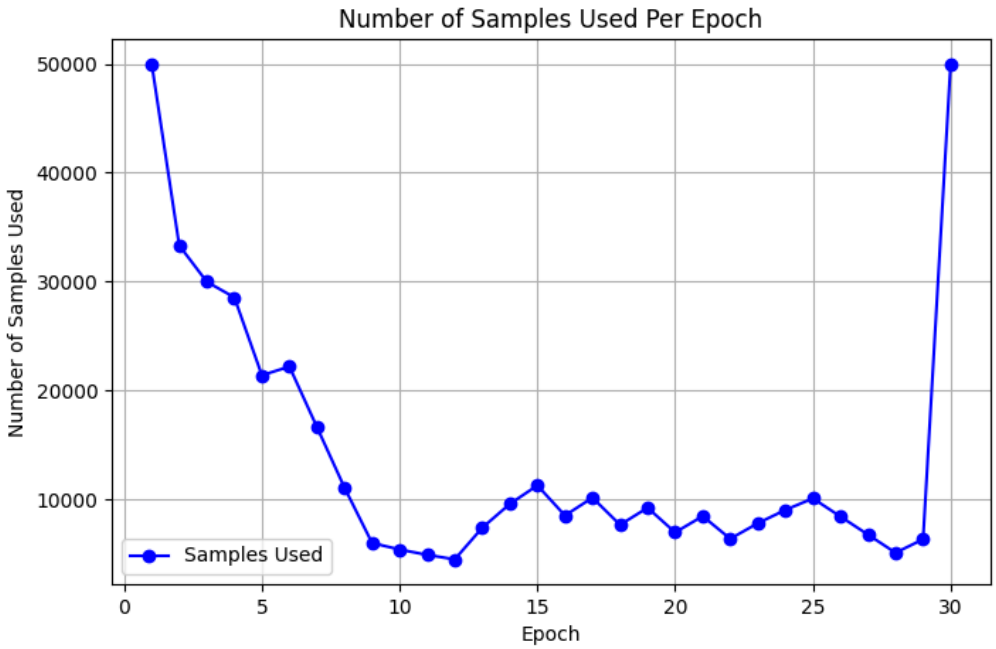
[SA] δ=+0.01 r=- boltz=- T=0.0117 → ACCEPT (improvement)
[SA] proposed α=2.10 accepted
Epoch time: 73.15s

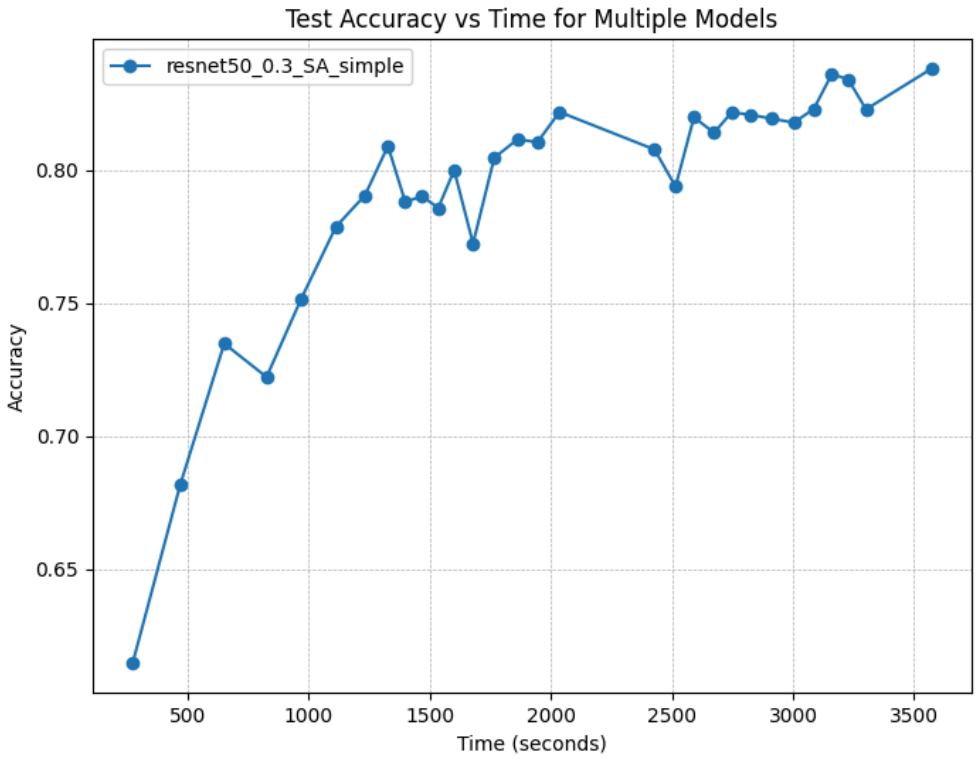
Epoch [30/30] α=2.10 T=0.0100 subset=50000
TrainAcc=0.91 TrainLoss=0.2568 | TestAcc=0.8382 TestLoss=0.5486

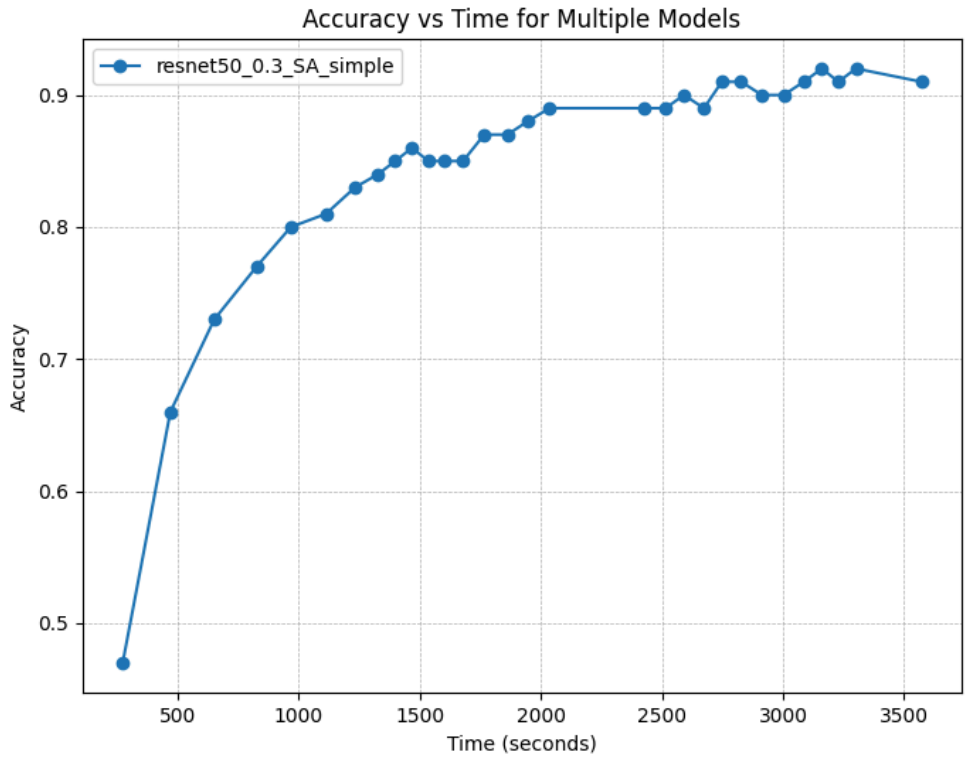
Epoch time: 270.89s
Training Time: 3572.51 seconds

Memory Consumption: 2956.92 MB

Total backprop samples: 424291
Effective epochs (float): 8.486
Info: JSON file 'cifar10_results_30epochs/resnet50' not found. Initializing fresh data.
Info: Data for 'resnet50_0.3_SA_simple' saved to 'cifar10_results_30epochs/resnet50'.
Info: Plot saved to 'cifar10_results_30epochs/resnet50'.
Info: Test data JSON file 'cifar10_results_30epochs/resnet50_test' not found. Initializing fresh data.
Info: Test data for 'resnet50_0.3_SA_simple' saved to 'cifar10_results_30epochs/resnet50_test'.
Info: Test accuracy plot saved to 'cifar10_results_30epochs/resnet50_test'.
Info: Epoch samples plot saved to 'cifar10_results_30epochs/resnet50_test_epochs_00.png'.
Number of steps: 424291
Effective Epochs: 8




Test Accuracy vs Time for Multiple Models
resnet50_0.3_SA_simple
Accuracy
0.80
0.75
0.70
0.65
500
1000
1500
2000
2500
3000
3500
Time (seconds)



train_with_sa 30 epochs resnet 50

(venv) ➜ energy_eff_training python ./main.py \
--model resnet50 \

--mode train_with_sa \

--epoch 30 \
--save_path cifar10_results_30epochs/resnet50 \

--dataset cifar10 \
--batch_size 32 \
--start_revision 0 \
--task classification \
--threshold 0.3 \
--seed 123
Using device: mps
Apple Silicon GPU (MPS) enabled.
/Users/amar/Developer/energy_eff_training/venv/lib/python3.13/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)

entry = pickle.load(f, encoding="latin1")
[DATA] Epoch 1: Jaccard(prev_subset) = N/A (first epoch)

Epoch [1/30] T_epoch=1.0000 subset=50000
Training: 0%|
| 0/1563 [00:00<?, ?it/s]/Users/amar/Developer/energy_eff_training/venv/lib/python3.13/site-packages/torch/utils/data/dataloader.py:1118: UserWarning: 'pin_memory' argument is set as true but not supported on MPS now, device pinned memory won't be used.
super().**init**(loader)
TrainAcc=0.4561 TestAcc=0.5622

[SA][ACCEPT:threshold] epoch=1 delta=+0.4561 T: 1.0000 → 0.5607
Epoch time: 271.0235s
[DATA] Epoch 2: Jaccard(prev_subset) = 0.5607 (|curr|=28035, |prev|=50000, |∩|=28035)

Epoch [2/30] T_epoch=0.5607 subset=28035
TrainAcc=0.6407 TestAcc=0.6640

[SA][ACCEPT:threshold] epoch=2 delta=+0.1846 T: 0.5607 → 0.4049
Epoch time: 172.5188s
[DATA] Epoch 3: Jaccard(prev_subset) = 0.3086 (|curr|=20245, |prev|=28035, |∩|=11387)

Epoch [3/30] T_epoch=0.4049 subset=20245
TrainAcc=0.6973 TestAcc=0.6974

[SA][ACCEPT:threshold] epoch=3 delta=+0.0566 T: 0.4049 → 0.3224
Epoch time: 136.3694s
[DATA] Epoch 4: Jaccard(prev_subset) = 0.2163 (|curr|=16120, |prev|=20245, |∩|=6466)

Epoch [4/30] T_epoch=0.3224 subset=16120
TrainAcc=0.7401 TestAcc=0.7241

[SA][ACCEPT:threshold] epoch=4 delta=+0.0428 T: 0.3224 → 0.2702
Epoch time: 117.1746s
[DATA] Epoch 5: Jaccard(prev_subset) = 0.1743 (|curr|=13510, |prev|=16120, |∩|=4399)

Epoch [5/30] T_epoch=0.2702 subset=13510
TrainAcc=0.7551 TestAcc=0.7275

[SA][ACCEPT:threshold] epoch=5 delta=+0.0150 T: 0.2702 → 0.2337
Epoch time: 105.7148s
[DATA] Epoch 6: Jaccard(prev_subset) = 0.1455 (|curr|=11685, |prev|=13510, |∩|=3201)

Epoch [6/30] T_epoch=0.2337 subset=11685
TrainAcc=0.7739 TestAcc=0.7365

[SA][ACCEPT:threshold] epoch=6 delta=+0.0188 T: 0.2337 → 0.2065
Epoch time: 97.2206s
[DATA] Epoch 7: Jaccard(prev_subset) = 0.1204 (|curr|=10325, |prev|=11685, |∩|=2365)

Epoch [7/30] T_epoch=0.2065 subset=10325
TrainAcc=0.7831 TestAcc=0.7419

[SA][ACCEPT:threshold] epoch=7 delta=+0.0092 T: 0.2065 → 0.1852
Epoch time: 90.9118s
[DATA] Epoch 8: Jaccard(prev_subset) = 0.1057 (|curr|=9260, |prev|=10325, |∩|=1873)

Epoch [8/30] T_epoch=0.1852 subset=9260
TrainAcc=0.7981 TestAcc=0.7468

[SA][ACCEPT:threshold] epoch=8 delta=+0.0150 T: 0.1852 → 0.1681
Epoch time: 86.8969s
[DATA] Epoch 9: Jaccard(prev_subset) = 0.0943 (|curr|=8405, |prev|=9260, |∩|=1522)

Epoch [9/30] T_epoch=0.1681 subset=8405
TrainAcc=0.8048 TestAcc=0.7647

[SA][ACCEPT:threshold] epoch=9 delta=+0.0067 T: 0.1681 → 0.1538
Epoch time: 82.5219s
[DATA] Epoch 10: Jaccard(prev_subset) = 0.0873 (|curr|=7690, |prev|=8405, |∩|=1292)

Epoch [10/30] T_epoch=0.1538 subset=7690
TrainAcc=0.8139 TestAcc=0.7887

[SA][ACCEPT:threshold] epoch=10 delta=+0.0091 T: 0.1538 → 0.1418
Epoch time: 79.0751s
[DATA] Epoch 11: Jaccard(prev_subset) = 0.0809 (|curr|=7090, |prev|=7690, |∩|=1106)

Epoch [11/30] T_epoch=0.1418 subset=7090
TrainAcc=0.8135 TestAcc=0.7685

[SA][DEBUG] epoch=11 delta=-0.0004, T=0.1418, r=0.6824, boltz_prob_raw=0.9930, accept_prob=0.9930, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=11 delta=-0.0004 T: 0.1418 → 0.1314
Epoch time: 76.4957s
[DATA] Epoch 12: Jaccard(prev_subset) = 0.0730 (|curr|=6570, |prev|=7090, |∩|=929)

Epoch [12/30] T_epoch=0.1314 subset=6570
TrainAcc=0.8257 TestAcc=0.7931

[SA][ACCEPT:threshold] epoch=12 delta=+0.0122 T: 0.1314 → 0.1224
Epoch time: 73.9870s
[DATA] Epoch 13: Jaccard(prev_subset) = 0.0664 (|curr|=6120, |prev|=6570, |∩|=790)

Epoch [13/30] T_epoch=0.1224 subset=6120
TrainAcc=0.8387 TestAcc=0.7690

[SA][ACCEPT:threshold] epoch=13 delta=+0.0130 T: 0.1224 → 0.1144
Epoch time: 72.2238s
[DATA] Epoch 14: Jaccard(prev_subset) = 0.0612 (|curr|=5720, |prev|=6120, |∩|=683)

Epoch [14/30] T_epoch=0.1144 subset=5720
TrainAcc=0.8467 TestAcc=0.7795

[SA][ACCEPT:threshold] epoch=14 delta=+0.0080 T: 0.1144 → 0.1073
Epoch time: 69.9785s
[DATA] Epoch 15: Jaccard(prev_subset) = 0.0600 (|curr|=5365, |prev|=5720, |∩|=627)

Epoch [15/30] T_epoch=0.1073 subset=5365
TrainAcc=0.8475 TestAcc=0.7636

[SA][DEBUG] epoch=15 delta=+0.0008, T=0.1073, r=0.0538, boltz_prob_raw=1.0188, accept_prob=1.0188, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=15 delta=+0.0008 T: 0.1073 → 0.1009
Epoch time: 68.4736s
[DATA] Epoch 16: Jaccard(prev_subset) = 0.0555 (|curr|=5045, |prev|=5365, |∩|=547)

Epoch [16/30] T_epoch=0.1009 subset=5045
TrainAcc=0.8460 TestAcc=0.7740

[SA][DEBUG] epoch=16 delta=-0.0015, T=0.1009, r=0.2204, boltz_prob_raw=0.9635, accept_prob=0.9635, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=16 delta=-0.0015 T: 0.1009 → 0.0952
Epoch time: 67.2864s
[DATA] Epoch 17: Jaccard(prev_subset) = 0.0510 (|curr|=4760, |prev|=5045, |∩|=476)

Epoch [17/30] T_epoch=0.0952 subset=4760
TrainAcc=0.8498 TestAcc=0.7877

[SA][DEBUG] epoch=17 delta=+0.0038, T=0.0952, r=0.1844, boltz_prob_raw=1.1049, accept_prob=1.1049, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=17 delta=+0.0038 T: 0.0952 → 0.0899
Epoch time: 65.7097s
[DATA] Epoch 18: Jaccard(prev_subset) = 0.0483 (|curr|=4495, |prev|=4760, |∩|=426)

Epoch [18/30] T_epoch=0.0899 subset=4495
TrainAcc=0.8536 TestAcc=0.7917

[SA][DEBUG] epoch=18 delta=+0.0038, T=0.0899, r=0.1759, boltz_prob_raw=1.1115, accept_prob=1.1115, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=18 delta=+0.0038 T: 0.0899 → 0.0852
Epoch time: 64.7172s
[DATA] Epoch 19: Jaccard(prev_subset) = 0.0433 (|curr|=4260, |prev|=4495, |∩|=363)

Epoch [19/30] T_epoch=0.0852 subset=4260
TrainAcc=0.8535 TestAcc=0.7949

[SA][DEBUG] epoch=19 delta=-0.0001, T=0.0852, r=0.8121, boltz_prob_raw=0.9971, accept_prob=0.9971, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=19 delta=-0.0001 T: 0.0852 → 0.0807
Epoch time: 63.9998s
[DATA] Epoch 20: Jaccard(prev_subset) = 0.0431 (|curr|=4035, |prev|=4260, |∩|=343)

Epoch [20/30] T_epoch=0.0807 subset=4035
TrainAcc=0.8575 TestAcc=0.7972

[SA][DEBUG] epoch=20 delta=+0.0040, T=0.0807, r=0.9233, boltz_prob_raw=1.1319, accept_prob=1.1319, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=20 delta=+0.0040 T: 0.0807 → 0.0767

Epoch time: 62.4553s
[DATA] Epoch 21: Jaccard(prev_subset) = 0.0410 (|curr|=3835, |prev|=4035, |∩|=310)

Epoch [21/30] T_epoch=0.0767 subset=3835
TrainAcc=0.8675 TestAcc=0.7657

[SA][ACCEPT:threshold] epoch=21 delta=+0.0100 T: 0.0767 → 0.0729
Epoch time: 61.6867s
[DATA] Epoch 22: Jaccard(prev_subset) = 0.0357 (|curr|=3645, |prev|=3835, |∩|=258)

Epoch [22/30] T_epoch=0.0729 subset=3645
TrainAcc=0.8617 TestAcc=0.7880

[SA][DEBUG] epoch=22 delta=-0.0058, T=0.0729, r=0.2766, boltz_prob_raw=0.8196, accept_prob=0.8196, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=22 delta=-0.0058 T: 0.0729 → 0.0693
Epoch time: 60.8562s
[DATA] Epoch 23: Jaccard(prev_subset) = 0.0339 (|curr|=3465, |prev|=3645, |∩|=233)

Epoch [23/30] T_epoch=0.0693 subset=3465
TrainAcc=0.8629 TestAcc=0.8152

[SA][DEBUG] epoch=23 delta=+0.0012, T=0.0693, r=0.8198, boltz_prob_raw=1.0442, accept_prob=1.0442, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=23 delta=+0.0012 T: 0.0693 → 0.0661
Epoch time: 60.2226s
[DATA] Epoch 24: Jaccard(prev_subset) = 0.0363 (|curr|=3305, |prev|=3465, |∩|=237)

Epoch [24/30] T_epoch=0.0661 subset=3305
TrainAcc=0.8766 TestAcc=0.8048

[SA][ACCEPT:threshold] epoch=24 delta=+0.0137 T: 0.0661 → 0.0630
Epoch time: 59.2174s
[DATA] Epoch 25: Jaccard(prev_subset) = 0.0336 (|curr|=3150, |prev|=3305, |∩|=210)

Epoch [25/30] T_epoch=0.0630 subset=3150
TrainAcc=0.8600 TestAcc=0.8035

[SA][DEBUG] epoch=25 delta=-0.0166, T=0.0630, r=0.8899, boltz_prob_raw=0.5175, accept_prob=0.5175, scalar=2.5000
[SA][REHEAT] epoch=25 delta=-0.0166 T: 0.0630 → 0.1575
Epoch time: 58.7174s
[DATA] Epoch 26: Jaccard(prev_subset) = 0.0481 (|curr|=7875, |prev|=3150, |∩|=506)

Epoch [26/30] T_epoch=0.1575 subset=7875
TrainAcc=0.8734 TestAcc=0.8042

[SA][ACCEPT:threshold] epoch=26 delta=+0.0134 T: 0.1575 → 0.0573
Epoch time: 79.9217s
[DATA] Epoch 27: Jaccard(prev_subset) = 0.0467 (|curr|=2865, |prev|=7875, |∩|=479)

Epoch [27/30] T_epoch=0.0573 subset=2865
TrainAcc=0.8841 TestAcc=0.8043

[SA][ACCEPT:threshold] epoch=27 delta=+0.0107 T: 0.0573 → 0.0548
Epoch time: 57.4056s
[DATA] Epoch 28: Jaccard(prev_subset) = 0.0277 (|curr|=2740, |prev|=2865, |∩|=151)

Epoch [28/30] T_epoch=0.0548 subset=2740
TrainAcc=0.8781 TestAcc=0.7886

[SA][DEBUG] epoch=28 delta=-0.0060, T=0.0548, r=0.5130, boltz_prob_raw=0.7605,
accept_prob=0.7605, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=28 delta=-0.0060 T: 0.0548 → 0.0523
Epoch time: 56.9309s
[DATA] Epoch 29: Jaccard(prev_subset) = 0.0239 (|curr|=2615, |prev|=2740, |∩|=125)

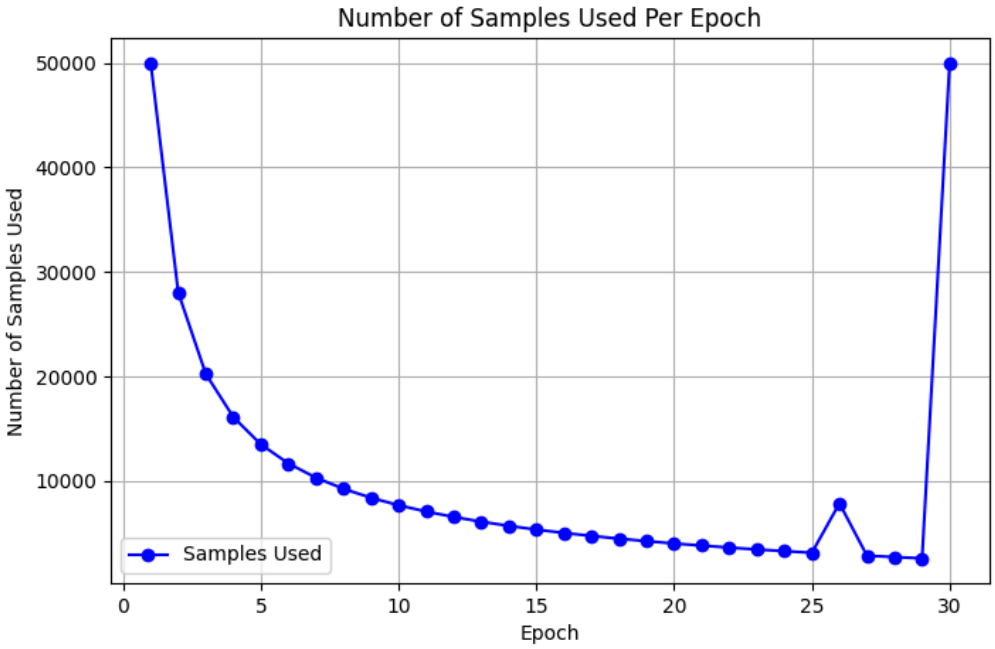
Epoch [29/30] T_epoch=0.0523 subset=2615
TrainAcc=0.8776 TestAcc=0.7970

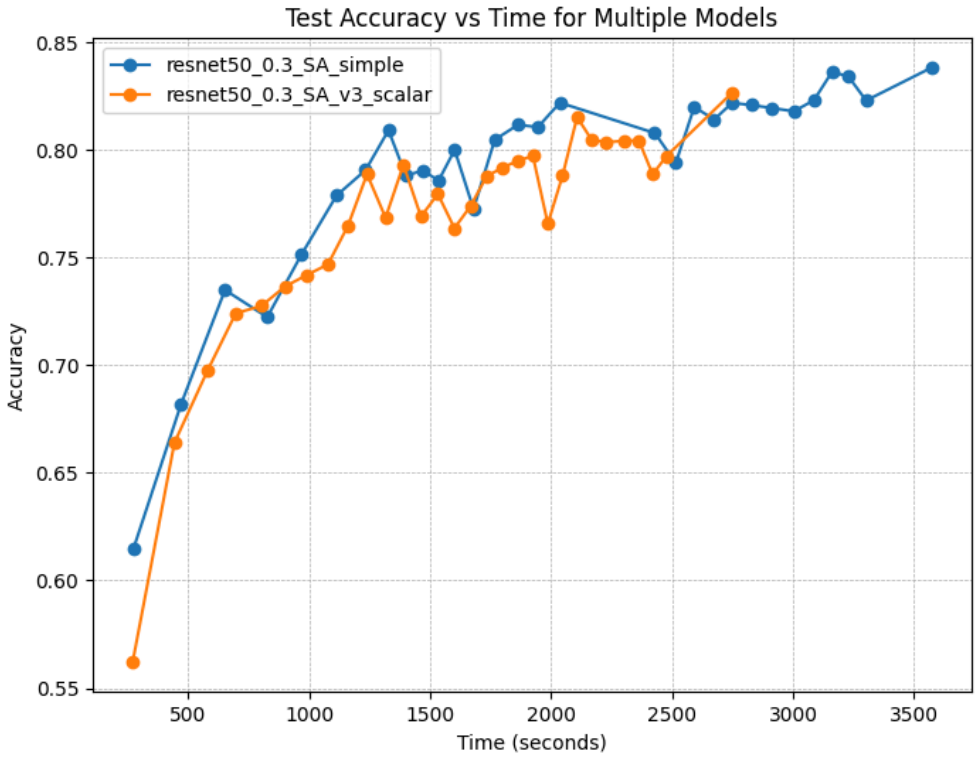
[SA][DEBUG] epoch=29 delta=-0.0005, T=0.0523, r=0.2450, boltz_prob_raw=0.9764,
accept_prob=0.9764, scalar=2.5000
[SA][ACCEPT:metropolis] epoch=29 delta=-0.0005 T: 0.0523 → 0.0500
Epoch time: 56.2910s
[DATA] Epoch 30: Jaccard(prev_subset) = 0.0523 (|curr|=50000, |prev|=2615, |∩|=2615)

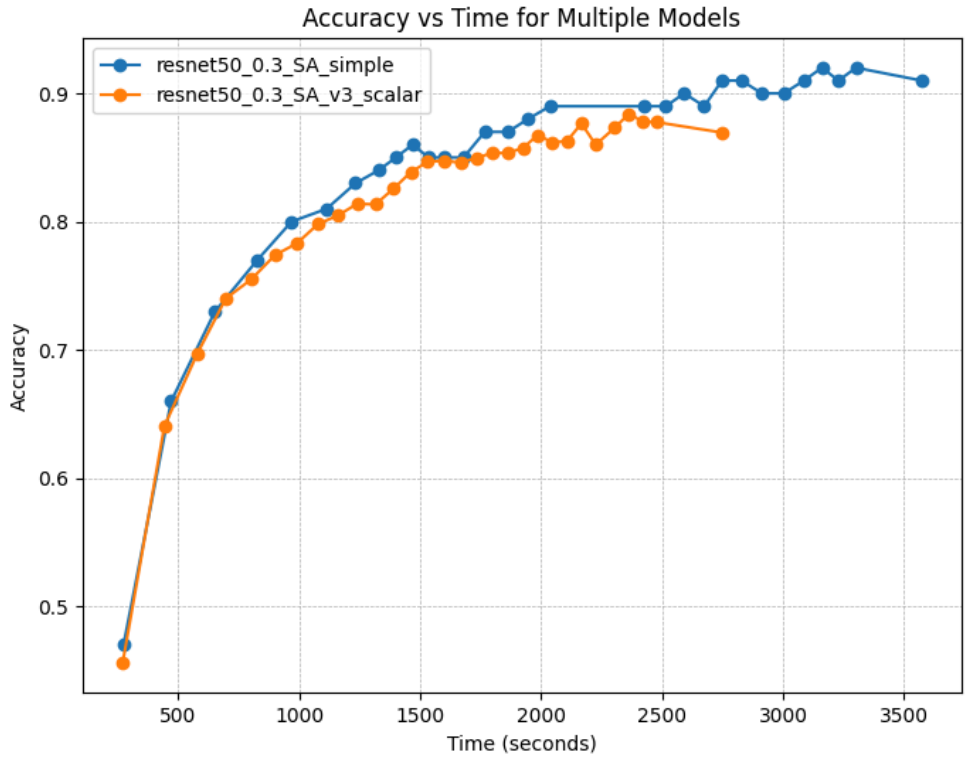
Epoch [30/30] T_epoch=1.0000 subset=50000
TrainAcc=0.8695 TestAcc=0.8264

Epoch time: 270.7995s
Training Time: 2746.80 seconds
Memory Consumption: 3457.38 MB

Total backprop samples: 312230
Effective epochs (float): 6.2446
Info: Data for 'resnet50_0.3_SA_v3_scalar' saved to 'cifar10_results_30epochs/resnet50'.
Info: Plot saved to 'cifar10_results_30epochs/resnet50'.
Info: Test data for 'resnet50_0.3_SA_v3_scalar' saved to 'cifar10_results_30epochs/resnet50_test'.
Info: Test accuracy plot saved to 'cifar10_results_30epochs/resnet50_test'.
Info: Epoch samples plot saved to 'cifar10_results_30epochs/resnet50_test_epochs_00.png'.
Number of steps: 312230
Effective Epochs: 6


Number of Samples Used Per Epoch
Number of Samples Used
Epoch
Samples Used
50000
40000
30000
20000
10000
0
5
10
15
20
25
30


Test Accuracy vs Time for Multiple Models
Accuracy
Time (seconds)
resnet50_0.3_SA_simple
resnet50_0.3_SA_v3_scalar
0.85
0.80
0.75
0.70
0.65
0.60
0.55
500
1000
1500
2000
2500
3000
3500



efficientnet_b0 — saRevisionSimple

Epoch	Test Accuracy	Si
1	0.6557	501
2	0.7309	333
3	0.7648	300
4	0.8010	285
5	0.7964	214
6	0.8135	222
7	0.8231	166
8	0.8288	111
9	0.8361	60
10	0.8366	54

mobilenet_v2 — saRevisionSimple

Epoch	Test Accuracy	Si
1	0.5930	501
2	0.6704	333
3	0.7099	300
4	0.7350	285
5	0.7627	214
6	0.7758	222
7	0.7745	166
8	0.7809	111
9	0.7967	60
10	0.7855	54

resnet50 — saRevisionSimple

Epoch	Test Accuracy	Si
1	0.6149	5(
2	0.6820	3:
3	0.7349	3(
4	0.7223	2{
5	0.7514	2'
6	0.7788	2:
7	0.7906	16
8	0.8091	11
9	0.7881	6(
10	0.7903	5{

Epoch	Test Accuracy	Su	Epoch	Test Accuracy	Su	Epoch	Test Accuracy	Su
11	0.8380	495	11	0.7689	495	11	0.7858	4
12	0.8254	454	12	0.7969	454	12	0.8000	4
13	0.8307	744	13	0.7946	744	13	0.7725	7
14	0.8390	962	14	0.8110	962	14	0.8047	9
15	0.8431	1131	15	0.8028	1131	15	0.8116	11
16	0.8453	856	16	0.8045	856	16	0.8106	8
17	0.8390	580	17	0.7990	1019	19	0.8219	1
18	0.8569	771	18	0.8108	771	18	0.8078	7
19	0.8433	521	19	0.8126	927	19	0.7939	9
20	0.8436	2736	20	0.8122	701	20	0.8199	7
21	0.8449	475	21	0.8193	850	26	0.8141	8
22	0.8533	643	22	0.8106	977	22	0.8218	6
23	0.8462	4356	23	0.8177	785	28	0.8208	7
24	0.8495	2282	24	0.8197	907	24	0.8194	9
25	0.8456	4025	25	0.8203	1012	25	0.8179	1
26	0.8465	2107	26	0.8177	1103	29	0.8230	8
27	0.8501	2029	27	0.8276	952	27	0.8362	6
28	0.8427	1957	28	0.8095	794	28	0.8341	5
29	0.8369	1892	29	0.8308	894	29	0.8229	6
30	0.8561	5030	30	0.8242	5030	30	0.8382	5

efficientnet_b0 — train_with_sa

mobilenet_v2 — train_with_sa

resnet50 — train_with_sa

Epoch	Test Accuracy	Su	Epoch	Test Accuracy	Su	Epoch	Test Accuracy	Su
1	0.6566	5010	00	0.5967	5010	00	0.5622	5
2	0.7355	2820	35	0.6516	2820	35	0.6640	2
3	0.7612	2024	45	0.6991	2024	45	0.6974	2
4	0.7797	1614	20	0.7111	1614	20	0.7241	1
5	0.7876	1355	10	0.7097	1355	10	0.7275	1
6	0.7965	1166	85	0.7391	1166	85	0.7365	1
7	0.8035	1037	25	0.7491	1037	25	0.7419	1
8	0.8090	926	80	0.7556	926	80	0.7468	9
9	0.8107	849	95	0.7620	849	95	0.7647	8
10	0.8188	769	10	0.7559	769	10	0.7887	7
11	0.8193	709	10	0.7683	709	10	0.7685	7

Epoch	Test Accuracy	Su	Epoch	Test Accuracy	Su	Epoch	Test Accuracy	Su
12	0.8208	6570	12	0.7801	6570	12	0.7931	65
13	0.8270	6120	13	0.7706	6120	13	0.7690	61
14	0.8251	5720	14	0.7876	5720	14	0.7795	57
15	0.8263	5365	15	0.7794	5365	15	0.7636	53
16	0.8242	5045	16	0.7837	5045	16	0.7740	50
17	0.8268	4760	17	0.7657	4760	17	0.7877	47
18	0.8311	4495	18	0.7742	4495	18	0.7917	44
19	0.8260	4260	19	0.7905	4260	19	0.7949	42
20	0.8212	4035	20	0.7564	4035	20	0.7972	40
21	0.8312	10025	21	0.7932	3825	21	0.7657	38
22	0.8316	3645	22	0.7941	3645	22	0.7880	36
23	0.8461	3465	23	0.7947	3465	23	0.8152	34
24	0.8431	3304	24	0.7949	3304	24	0.8048	33
25	0.8353	3155	25	0.7872	3155	25	0.8035	31
26	0.8348	7875	26	0.7910	3025	26	0.8042	78
27	0.8427	2867	27	0.7983	2867	27	0.8043	28
28	0.8475	2748	28	0.7927	2748	28	0.7886	27
29	0.8380	2639	29	0.7938	2639	29	0.7970	26
30	0.8528	50030	30	0.8079	50030	30	0.8264	50